2024311101-赵苏亚-机器人、遥操作控制器开源技术报告

1.1 机器人核心参数

名称	型号	数量	参数	用途
重量、重心			35kg	
尺寸（长宽高）				
传感器	ATK-MS530L0M激光测距模块	2	测量距离：40~1600mm 分辨率：1mm	测定底盘离地距离，实现自动上台阶
	MT6701磁编码器	3	反馈数据精度：14bit 系统延时：5us	测量遥操作控制器rpy云台反馈角
工作时电压			24V	
电机	MG6010E-i6-v3	1	重量：343g 额定转矩：5nm	机械臂joint1
	MG8016-i6-v3	1	重量：759g 额定转矩：12nm	机械臂joint2
	DM8009-2EC	1	重量：930g 额定转矩：20nm	机械臂joint3
	DM-J4310-2EC	1	重量：300g 额定转矩：3.5nm	机械臂joint4

	DM-J4310-2EC	1	重量：300g 额定转矩：3.5nm	机械臂joint5
	DM-S2325-1EC	1	重量：172.2g 额定转矩：1.35nm	机械臂joint6
	M3508	1	额定转矩：0.15nm	夹爪电机
	DM-8009p-2EC	8	重量：930g 额定转矩：20nm	轮足关节电机
	M3508	3	额定转矩：0.15nm	delta机械臂电机
电磁铁	LS-P20/15电磁铁	2	吸力：29.4N	能量单元存储机构
云台自由度			6自由度	
车体最大移动速度			3m/s	

2.1 控制框架与算法设计

2.1.1 系统架构

2.1.1.1 开发环境

本项目采用**现代化的嵌入式开发工作流**,基于 Linux 系统构建完整的开发生态。

核心开发工具

- **开发平台**: Linux 操作系统
- **集成开发环境**: Visual Studio Code (VSCode)
- **编程语言**: C/C++ (C23/C11 & C++20)
- **版本控制**: Git (GitLab)

嵌入式代码生成与工具链

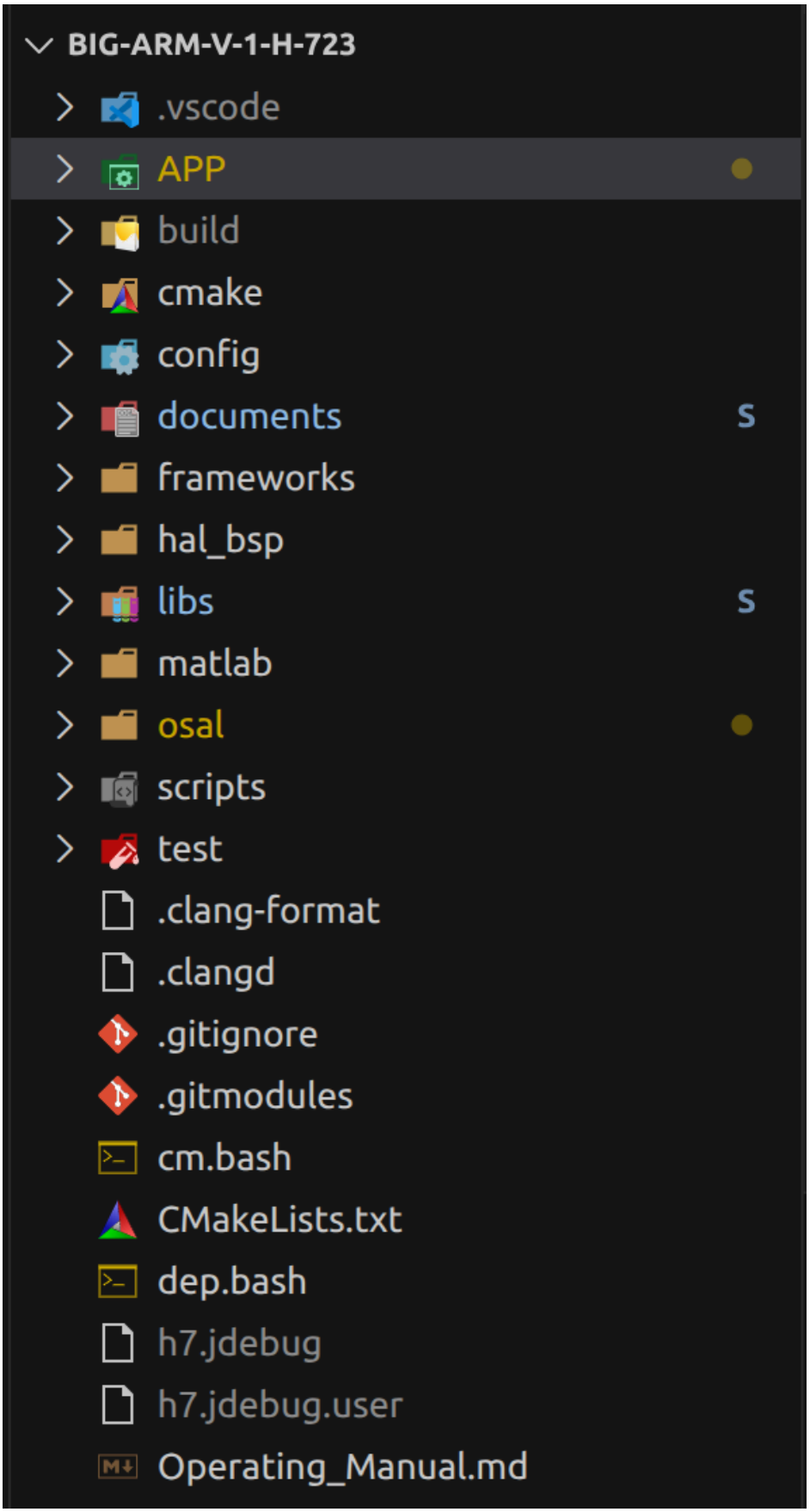
- **代码生成**

    - **STM32CubeMX**: 用于 STM32 微控制器的外设配置和代码自动生成,支持多种 MCU 板型
- **交叉编译工具链**
    - **编译工具链**: `arm-none-eabi-gcc/g++/ar` 等工具集
    - **目标架构**: ARM Cortex-M 系列处理器
- **编译流程架构**

  CMake 配置 → **Ninja** 构建 → arm-none-eabi-* 工具链 → 二进制文件生成
- **调试软件**
    - **ozone + systemview**

2.1.1.2 代码框架

机器人使用了哈尔滨工业大学（深圳）南工骁鹰战队基于25赛季电控框架改良的新框架，使用了自底向上的构建思路：

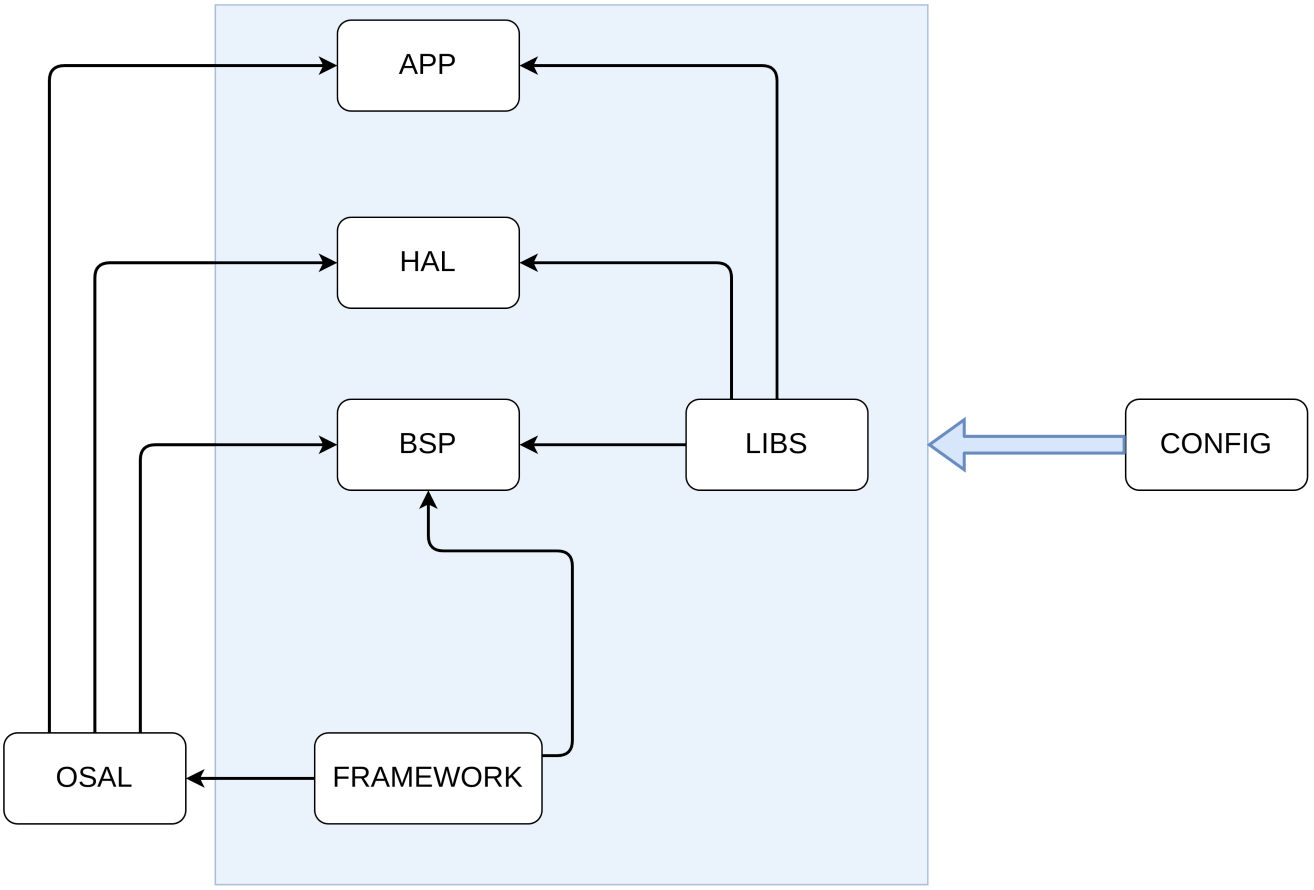


> 机器人电控代码根目录结构

主要分层:APP层、HAL层、BSP层、framework层、config层、osal层、libs层

config：“配置层”，通过cmake的头文件依赖管理功能，对其他各个层级的源代码进行宏定义注入，进而达到集中配置框架功能的目的，并在不同层级的配置宏之间实现隔离

- framework：主要包含stm32cubemx生成的不同型号芯片的裸机框架，主要负责对单片机的片上外设进行初始化提供hal库函数供上层使用、提供freertos的接口函数
- osal：“操作系统抽象层”，由于在实际使用freertos的过程中发现其具有过分轻量化的问题，缺少丰富的调试信息，故设计该层对操作系统的时基获取、任务创建调度等功能进行重定向，解除上层与freertos的耦合，便于日后更换更加完善的操作系统
- libs：“通用库”，部分功能上高度模块化的对象或外部库代码存放之处，提供诸如pid控制器、卡尔曼滤波器、机械臂连杆系等对象的相关api
- BSP：“板级支持包”，主要负责对stm32cubemx生成hal库函数进行初步封装，对片上外设全局对象进行重映射，为上层提供兼容不同型号芯片的通信api
- HAL：“硬件抽象层”，对物理世界的实体诸如电机、遥控器、裁判系统进行抽象，实现其驱动；对通信接口如uart、can等进行进一步封装，加入自定义通信协议
- APP：“应用层”，一般代码开发所在层级，主要是调用下层提供的api对机器人运行逻辑进行编写，同时是main函数与线程创建之处



> 电控框架依赖关系图

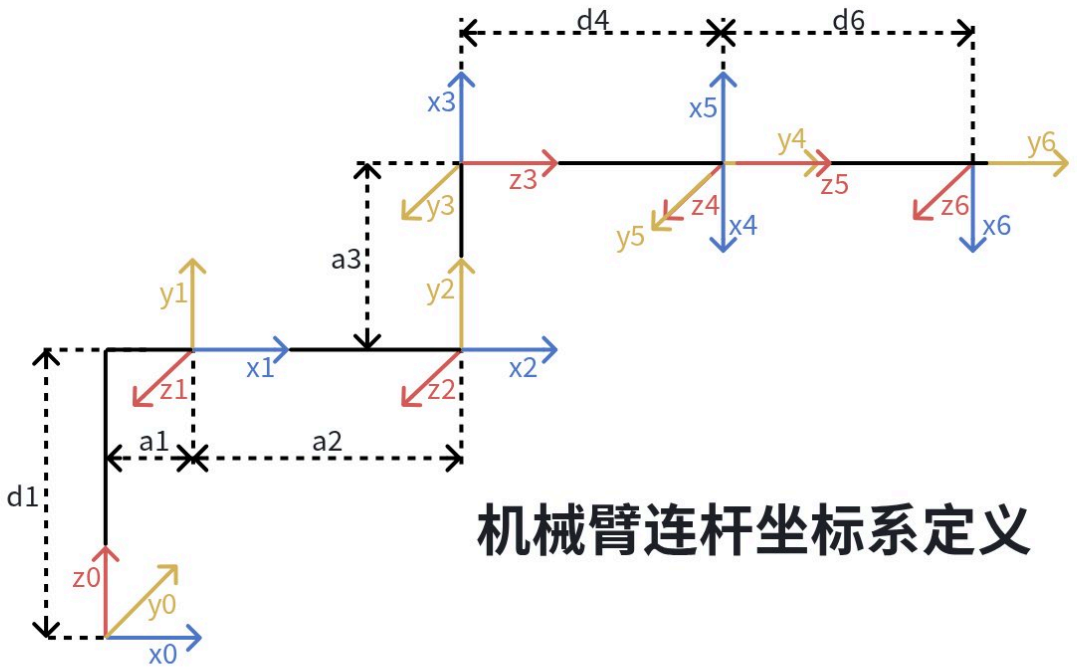
2.1.2 基础控制流程

2.1.2.1 机械臂

为了实现机械臂的基础运动功能，我们使用的控制算法为单关节位置速度串级环PID+重力前馈控制，其中重力前馈控制的实现依赖于连杆系物理建模，具体计算方法为递归牛顿欧拉法。

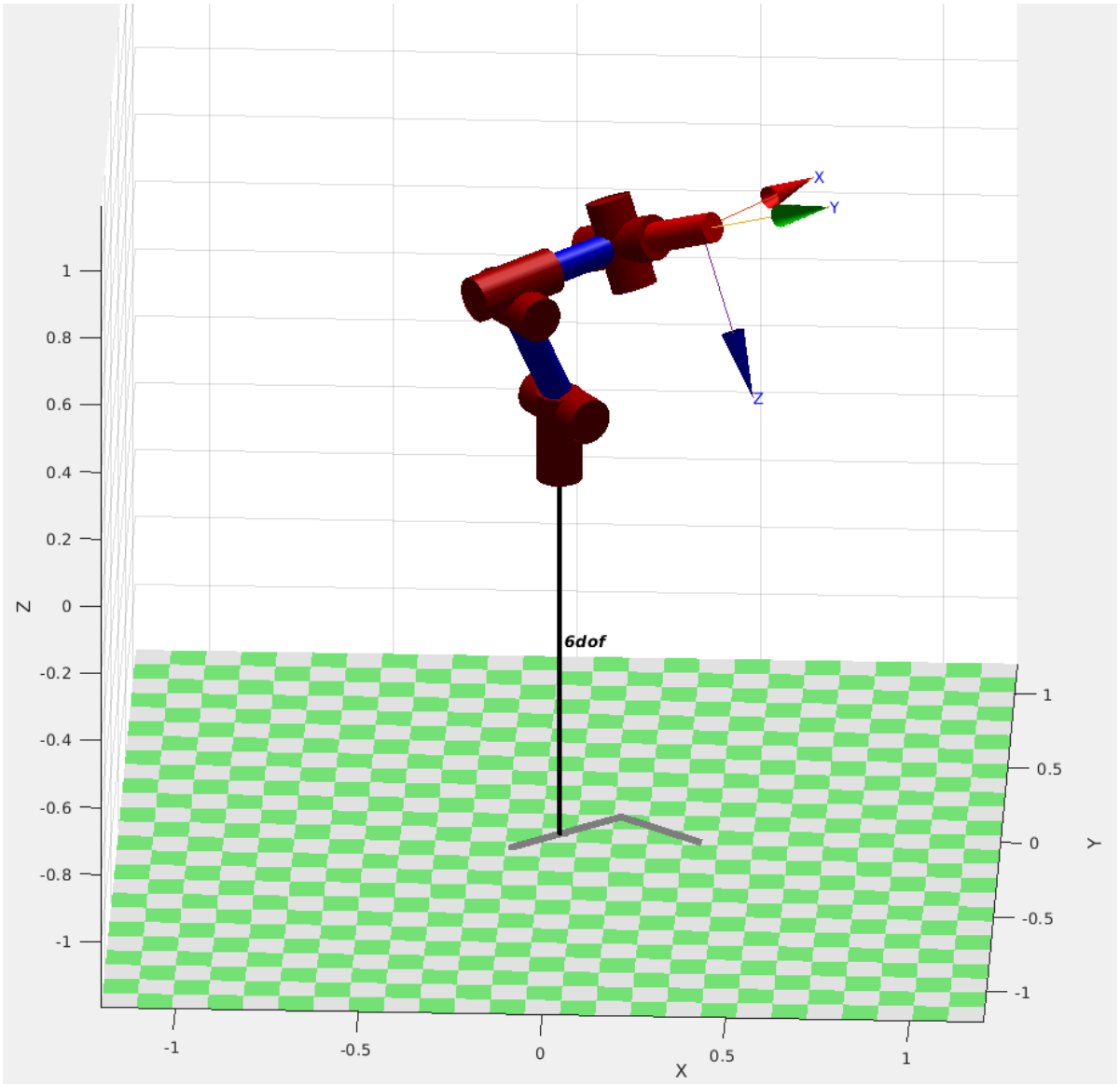
2.1.2.1.1 连杆系建模

首先我们使用标准DH法对机械臂的连杆系进行建模：

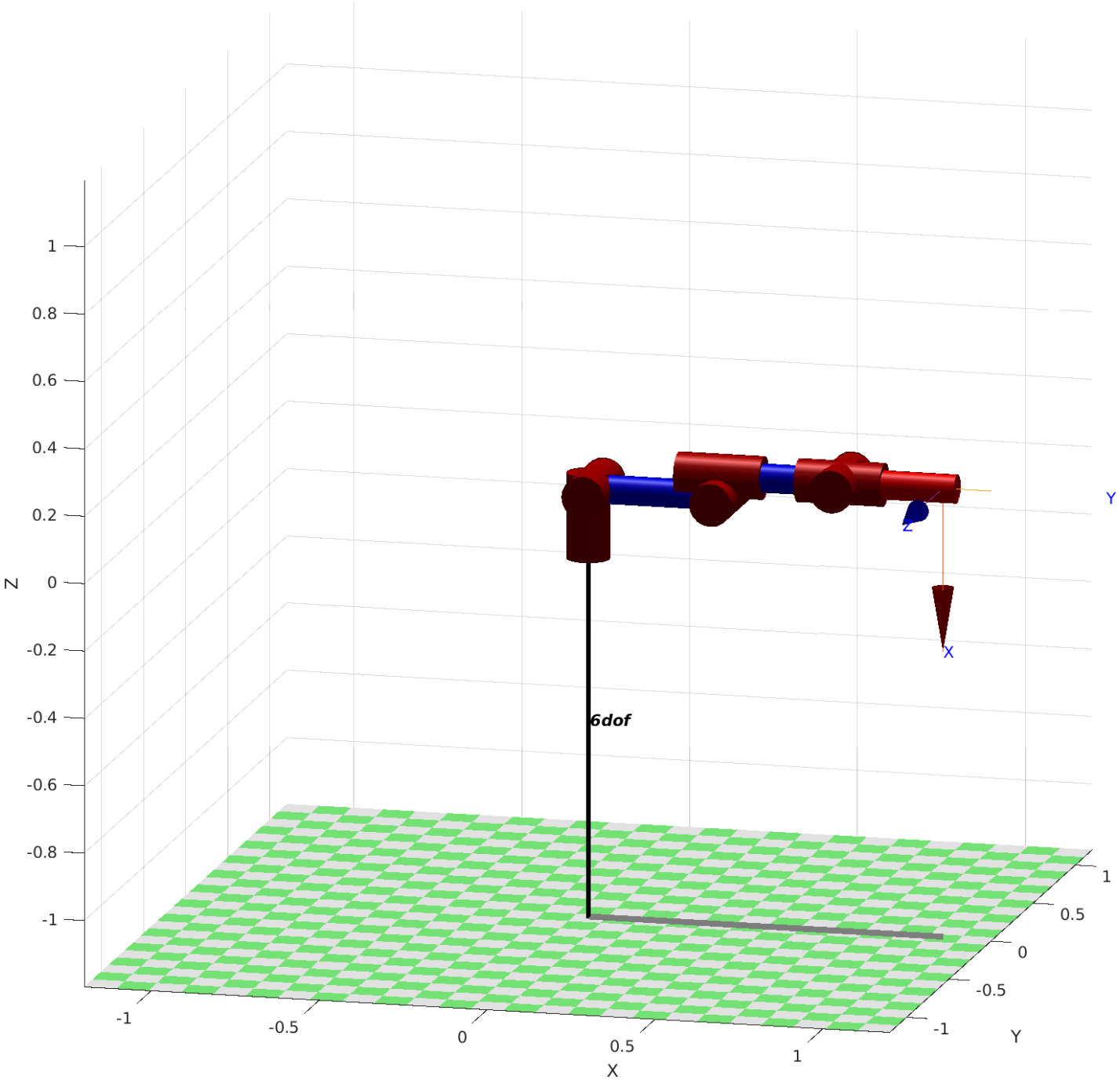


> 连杆坐标系定义

我们所定义的六轴机械臂在matlab中使用机器人学工具箱进行可视化的效果如下：



定义连杆系的零点位置（当关节变量 q 为**0**向量时）如下所示：



零点定义

2.1.2.1.2 递归牛顿欧拉法（RNE）

10.5 递归牛顿－欧拉动力学建模方法

前面推导的拉格朗日动力学方程具有存在解析表达式、物理意义明显的优点，但计算量较大，不适用于自由度多、系统复杂、进行实时计算的场合。采用递推的方法进行动力学求解，则计算效率高，实时性好，特别适用于计算机编程实现。本节介绍基于牛顿方程和欧拉方程的机器人系统动力学建模方法，称为牛顿－欧拉递推动力学建模方法。

10.5.1 连杆运动及受力情况分析

连杆 i 的运动及受力情况对图 10.11 所示。

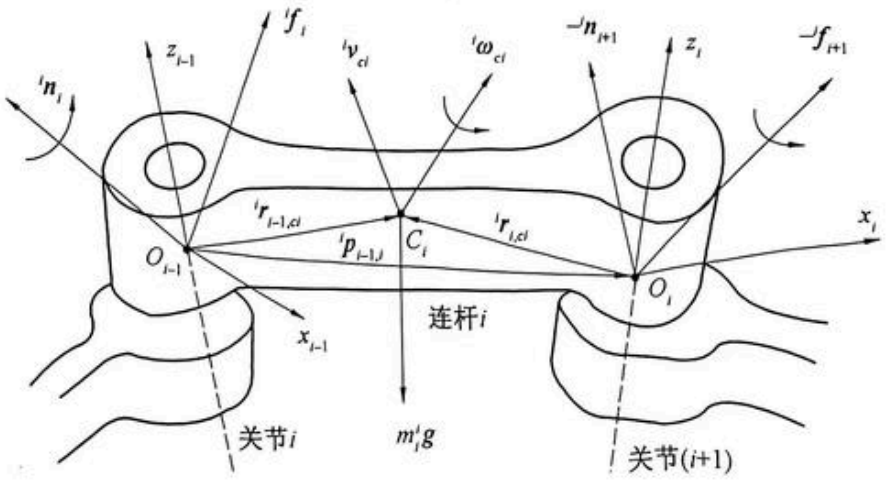


图 10.11 连杆 i 的运动及受力情况

各符号及变量的定义如下。

(1) 坐标系$\{O_{i-1}x_{i-1}y_{i-1}z_{i-1}\}$、坐标系$\{O_{i}x_{i}y_{i}z_{i}\}$ 分别为连杆$(i-1)$ 和连杆 i 的坐标系，按 D－H 规则建立，原点分别为 O_{i-1} 和 O_{i}。

(2)C_{i} 为连杆 i 的质心，m_{i} 为连杆 i 的质量，$^{i}\boldsymbol{I}_{ci}$ 为连杆 i 相对于自身质心的惯量矩阵，在$\{i\}$ 中的表示。

$^{i}\boldsymbol{f}_{i}$ 表示杆件$(i-1)$ 对杆件 i 所施加的力，在坐标系$\{i\}$ 中的表示。

$^{i}\boldsymbol{f}_{i+1}$ 表示杆件 i 对杆件$(i+1)$ 所施加的力，在坐标系$\{i\}$ 中的表示。

$^{i}\boldsymbol{n}_{i}$ 表示杆件$(i-1)$ 对杆件 i 所施加的力矩，在坐标系$\{i\}$ 中的表示。

${}^{i}\boldsymbol{n}_{i+1}$ 表示杆件 i 对杆件$(i+1)$ 所施加的力矩，在坐标系$\{i\}$ 中的表示。

${}^{i}\boldsymbol{r}_{i-1,a}$ 表示原点 O_{i-1} 到连杆质心 C_i 的位置矢量，在坐标系$\{i\}$ 中表示。

${}^{i}\boldsymbol{r}_{i,a}$ 表示原点 O_i 到连杆质心 C_i 的位置矢量，在坐标系$\{i\}$ 中表示。

${}^{i}\boldsymbol{p}_{i-1,i}$ 表示原点 O_{i-1} 到原点 O_i 的位置矢量，在坐标系$\{i\}$ 中表示。

${}^{i}\boldsymbol{v}_{a}$ 表示连杆 i 质心的线速度矢量，在坐标系$\{i\}$ 中表示。

${}^{i}\boldsymbol{\omega}_{a}$ 表示连杆 i 的角速度矢量，在坐标系$\{i\}$ 中表示。

$m_i{}^{i}\boldsymbol{g}$ 表示杆件 i 所受的重力，在坐标系$\{i\}$ 中表示。

10.5.2　杆件的力和力矩平衡方程

以$\{i\}$ 系为参考系，以连杆 i 的质心为作用点，建立力系的平衡方程。

首先，根据牛顿方程，可得连杆 i 的力平衡方程，即：

$$ {}^{i}\boldsymbol{f}_{a}={}^{i}\boldsymbol{f}_{i}-{}^{i}\boldsymbol{f}_{i+1}+m_i{}^{i}\boldsymbol{g} \tag{10.143}$$

式中，${}^{i}\boldsymbol{f}_{a}$ 为杆件 i 在质心处所受的合外力。

由于重力为有势力，其大小仅与臂型有关而与广义坐标的速度、加速度无关，因此，可将式(10.143) 中杆件的自重项 $m_i{}^{i}\boldsymbol{g}$ 先忽略掉，而其作用可通过将基座(即连杆 0) 的初始加速度设置为重力加速度($\dot{\boldsymbol{v}}_0=\boldsymbol{g}$) 来等效。此时方程可写成如下形式：

$$ {}^{i}\boldsymbol{f}_{a}={}^{i}\boldsymbol{f}_{i}-{}^{i}\boldsymbol{f}_{i+1} \tag{10.144}$$

另一方面，根据欧拉方程，可得连杆 i 的力矩平衡方程，即：

$$ {}^{i}\boldsymbol{n}_{a}={}^{i}\boldsymbol{n}_{i}-{}^{i}\boldsymbol{n}_{i+1}+{}^{i}\boldsymbol{f}_{i}\times{}^{i}\boldsymbol{r}_{i-1,a}-{}^{i}\boldsymbol{f}_{i+1}\times{}^{i}\boldsymbol{r}_{i,a} \tag{10.145}$$

式中，${}^{i}\boldsymbol{n}_{a}$ 表示杆件 i 所受的合外力矩(力系等效到了连杆质心处)。

根据牛顿第二定律可得，连杆 i 的合外力、外力矩与其加速度之间满足如下关系：

$$ \begin{cases} {}^{i}\boldsymbol{f}_{a}=m_i{}^{i}\dot{\boldsymbol{v}}_{a} \\ {}^{i}\boldsymbol{n}_{a}={}^{i}\boldsymbol{I}_{a}{}^{i}\dot{\boldsymbol{\omega}}_{i}+{}^{i}\boldsymbol{\omega}_{i}\times{}^{i}\boldsymbol{I}_{a}{}^{i}\boldsymbol{\omega}_{i} \end{cases} \tag{10.146}$$

其中，$\dot{\boldsymbol{v}}_a$ 为杆件 i 质心处的线加速度(不含重力加速度)；$\boldsymbol{\omega}_i$ 为杆件 i 的角速度；$\dot{\boldsymbol{\omega}}$ 为角加速度。

10.5.3　速度及加速度递推方程

从运动的角度而言，连杆 0(即基座) 的边界条件已知，因此，采用外推的方法，从连杆 0 的运动外推到连杆 n，可以依次计算得到各连杆的速度和加速度。

对于连杆 $i(i=1,\cdots,n)$，其角速度是连杆$(i-1)$ 的角速度与连杆 i 相对于连杆$(i-1)$ 的角速度之和。对于旋转关节，相对角速度即为关节角速度(方向沿 z_{i-1} 轴)；对于移动关节，连杆间的相对角速度为 0，因此，有如下关系($i=1,\cdots,n$)：

$$ {}^{i}\boldsymbol{\omega}_{i}=\begin{cases} {}^{i}\boldsymbol{R}_{i-1}({}^{i-1}\boldsymbol{\omega}_{i-1}+{}^{i-1}\boldsymbol{z}_{i-1}\dot{\boldsymbol{q}}_i) & \text{旋转关节} \\ {}^{i}\boldsymbol{R}_{i-1}{}^{i-1}\boldsymbol{\omega}_{i-1} & \text{移动关节} \end{cases} \tag{10.147}$$

类似地，连杆 i 坐标系原点 O_i 处的线速度，可以通过连杆$(i-1)$ 坐标系原点 O_{i-1} 处的线速度加上牵连运动引起的相对速度。对于旋转关节和平动关节，分别有如下关系：

$$ {}^{i}\boldsymbol{v}_{i}=\begin{cases} {}^{i}\boldsymbol{R}_{i-1}{}^{i-1}\boldsymbol{v}_{i-1}+{}^{i}\boldsymbol{\omega}_{i}\times{}^{i}\boldsymbol{p}_{i-1,i} & \text{旋转关节} \\ {}^{i}\boldsymbol{R}_{i-1}({}^{i-1}\boldsymbol{v}_{i-1}+{}^{i-1}\boldsymbol{z}_{i-1}\dot{\boldsymbol{q}}_i)+{}^{i}\boldsymbol{\omega}_{i}\times{}^{i}\boldsymbol{p}_{i-1,i} & \text{移动关节} \end{cases} \tag{10.148}$$

对式(10.147) 求导可得连杆 i 的角加速度为

$$ {}^{i}\dot{\boldsymbol{\omega}}_i=\begin{cases}{}^{i}\boldsymbol{R}_{i-1}\left({}^{i-1}\dot{\boldsymbol{\omega}}_{i-1}+{}^{i-1}\boldsymbol{z}_{i-1}\ddot{q}_i+{}^{i-1}\boldsymbol{\omega}_{i-1}\times{}^{i-1}\boldsymbol{z}_{i-1}\dot{q}_i\right) & \text{旋转关节}\\ {}^{i}\boldsymbol{R}_{i-1}\,{}^{i-1}\dot{\boldsymbol{\omega}}_{i-1} & \text{移动关节}\end{cases} \tag{10.149}$$

对式(10.148)求导可得连杆 i 坐标系原点 O_i 处的线加速度为

$$ {}^{i}\dot{\boldsymbol{v}}_i=\begin{cases}{}^{i}\boldsymbol{R}_{i-1}\,{}^{i-1}\dot{\boldsymbol{v}}_{i-1}+{}^{i}\dot{\boldsymbol{\omega}}_i\times{}^{i}\boldsymbol{p}_{i-1,i}+{}^{i}\boldsymbol{\omega}_i\times\left({}^{i}\boldsymbol{\omega}_i\times{}^{i}\boldsymbol{p}_{i-1,i}\right), & \text{旋转关节}\\ {}^{i}\boldsymbol{R}_{i-1}\left({}^{i-1}\dot{\boldsymbol{v}}_{i-1}+{}^{i-1}\boldsymbol{z}_{i-1}\ddot{q}_i\right)+{}^{i}\dot{\boldsymbol{\omega}}_i\times{}^{i}\boldsymbol{p}_{i-1,i}+{}^{i}\boldsymbol{\omega}_i\times\left({}^{i}\boldsymbol{\omega}_i\times{}^{i}\boldsymbol{p}_{i-1,i}\right)+ & \\ 2{}^{i}\boldsymbol{\omega}_i\times\left({}^{i}\boldsymbol{R}_{i-1}\,{}^{i-1}\boldsymbol{z}_{i-1}\dot{q}_i\right), & \text{移动关节}\end{cases} \tag{10.150}$$

基于 O_i 处的线加速度，可进一步得到连杆质心 C_i 处的线加速度为

$$ {}^{i}\dot{\boldsymbol{v}}_{ci}=\begin{cases}{}^{i}\dot{\boldsymbol{v}}_i+{}^{i}\dot{\boldsymbol{\omega}}_i\times{}^{i}\boldsymbol{r}_{i,ci}+{}^{i}\boldsymbol{\omega}_i\times\left({}^{i}\boldsymbol{\omega}_i\times{}^{i}\boldsymbol{r}_{i,ci}\right) & \text{旋转关节}\\ {}^{i}\dot{\boldsymbol{v}}_i & \text{移动关节}\end{cases} \tag{10.151}$$

对于基座固定的情况，上述迭代过程中的边界条件为 ${}^{0}\boldsymbol{\omega}_0=\boldsymbol{0}$，${}^{0}\dot{\boldsymbol{\omega}}_0=\boldsymbol{0}$，${}^{0}\boldsymbol{v}_0=\boldsymbol{0}$，${}^{0}\dot{\boldsymbol{v}}_0=-{}^{0}\boldsymbol{g}$。其中迭代中，设置 ${}^{0}\dot{\boldsymbol{v}}_0=-{}^{0}\boldsymbol{g}$，就将作用在连杆上的重力因素包含到动力学方程中。

10.5.4 力和力矩递推方程

对力和力矩平衡方程式(10.144)和式(10.145)进行重新排列，可得如下的递推关系(向内递推，$i:n\to 1$)：

$$ \begin{cases}{}^{i}\boldsymbol{f}_i={}^{i}\boldsymbol{f}_{i+1}+{}^{i}\boldsymbol{f}_{ci}={}^{i}\boldsymbol{R}_{i+1}\,{}^{i+1}\boldsymbol{f}_{i+1}+{}^{i}\boldsymbol{f}_{ci}\\ {}^{i}\boldsymbol{n}_i={}^{i}\boldsymbol{n}_{i+1}+{}^{i}\boldsymbol{n}_{ci}+{}^{i}\boldsymbol{f}_i\times{}^{i}\boldsymbol{r}_{i-1,ci}-{}^{i}\boldsymbol{f}_{i+1}\times{}^{i}\boldsymbol{r}_{i,ci}\\ \quad={}^{i}\boldsymbol{R}_{i+1}\,{}^{i+1}\boldsymbol{n}_{i+1}+{}^{i}\boldsymbol{n}_{ci}+{}^{i}\boldsymbol{f}_i\times{}^{i}\boldsymbol{r}_{i-1,ci}-\left({}^{i}\boldsymbol{R}_{i+1}\,{}^{i+1}\boldsymbol{f}_{i+1}\right)\times{}^{i}\boldsymbol{r}_{i,ci}\end{cases} \tag{10.152}$$

从末端连杆 n 开始向内迭代一直到连杆 1，即可以得到所有关节处的作用力 ${}^{i}\boldsymbol{f}_i$ 和力矩 ${}^{i}\boldsymbol{n}_i$，而关节的驱动力 / 力矩即为 ${}^{i}\boldsymbol{f}_i$ 或 ${}^{i}\boldsymbol{n}_i$ 沿运动轴的分量。

若关节 i 为旋转关节，其驱动力矩等于连杆$(i-1)$作用在连杆 i 上的作用力矩 ${}^{i}\boldsymbol{n}_i$ 沿关节 i 旋转轴(即 D－H 坐标系中的 z_{i-1} 轴)的分量，即：

$$ \tau_i={}^{i}\boldsymbol{n}_i^{\mathrm{T}}\,{}^{i}\boldsymbol{z}_{i-1}={}^{i}\boldsymbol{n}_i^{\mathrm{T}}\left({}^{i}\boldsymbol{R}_{i-1}\,{}^{i-1}\boldsymbol{z}_{i-1}\right)={}^{i}\boldsymbol{n}_i^{\mathrm{T}}\left({}^{i-1}\boldsymbol{R}_i^{\mathrm{T}}\boldsymbol{z}_0\right) \tag{10.153}$$

式中，${}^{i-1}\boldsymbol{z}_{i-1}=\boldsymbol{z}_0=[0,0,1]^{\mathrm{T}}$。

若关节 i 为移动关节，其驱动力等于连杆$(i-1)$作用在连杆 i 上的作用力 ${}^{i}\boldsymbol{f}_i$ 沿关节 i 移动轴的分量，即：

$$ \tau_i={}^{i}\boldsymbol{f}_i^{\mathrm{T}}\,{}^{i}\boldsymbol{z}_{i-1}={}^{i}\boldsymbol{f}_i^{\mathrm{T}}\left({}^{i}\boldsymbol{R}_{i-1}\,{}^{i-1}\boldsymbol{z}_{i-1}\right)={}^{i}\boldsymbol{f}_i^{\mathrm{T}}\left({}^{i-1}\boldsymbol{R}_i^{\mathrm{T}}\boldsymbol{z}_0\right) \tag{10.154}$$

将式(10.153)和式(10.154)组合，可得

$$ \tau_i=\begin{cases}{}^{i}\boldsymbol{n}_i^{\mathrm{T}}\left({}^{i-1}\boldsymbol{R}_i^{\mathrm{T}}\boldsymbol{z}_0\right), & \text{旋转关节}\\ {}^{i}\boldsymbol{f}_i^{\mathrm{T}}\left({}^{i-1}\boldsymbol{R}_i^{\mathrm{T}}\boldsymbol{z}_0\right), & \text{移动关节}\end{cases} \tag{10.155}$$

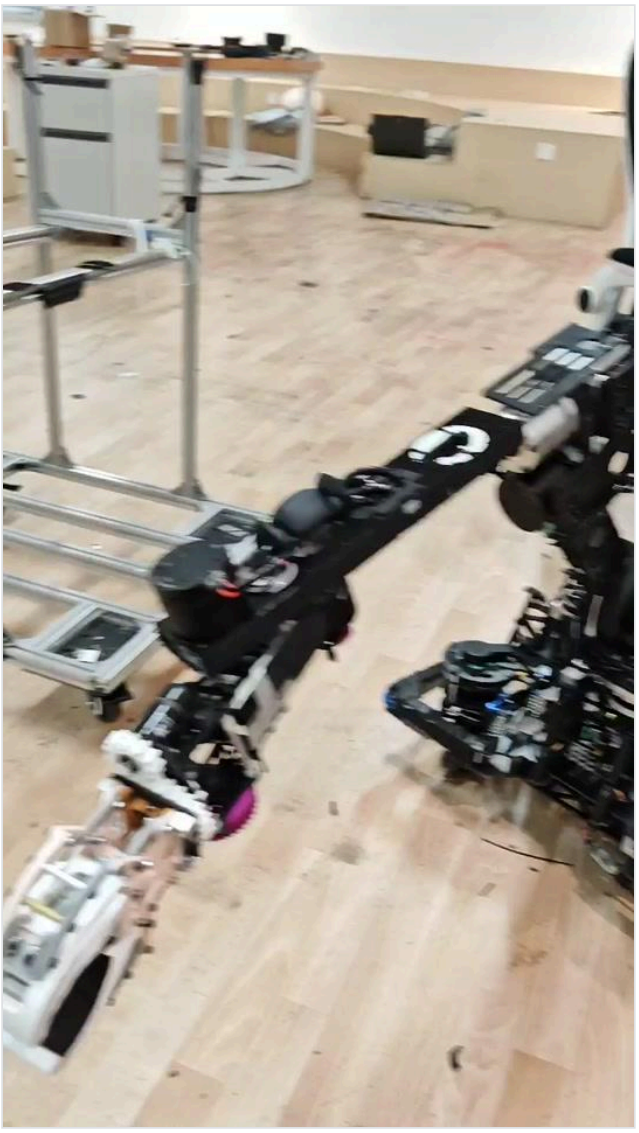
需要指出的是，在迭代中出现$(n+1)$杆的问题，相当于将环境当成了杆件$(n+1)$，其坐标系与$\langle n\rangle$系完全一致。相应地，递推初值 ${}^{n+1}\boldsymbol{f}_{n+1}$、${}^{n+1}\boldsymbol{n}_{n+1}$ 的规定如下：若机械臂末端(杆件 n)与环境(相当于杆件$(n+1)$)不接触，则 ${}^{n+1}\boldsymbol{f}_{n+1}=\boldsymbol{0}$、${}^{n+1}\boldsymbol{n}_{n+1}=\boldsymbol{0}$；若机械臂末端与环境接触时，${}^{n+1}\boldsymbol{f}_{n+1}={}^{n+1}\boldsymbol{f}_e$、${}^{n+1}\boldsymbol{n}_{n+1}={}^{n+1}\boldsymbol{m}_e$，其中 ${}^{n+1}\boldsymbol{f}_e$、${}^{n+1}\boldsymbol{n}_e$ 分别为末端杆件(杆件 n)对环境产生的作用力和作用力矩，即前述的末端操作力。

> RNE的公式推导，来自徐文福教授的《机器人学：基础理论与应用实践》

具体代码实现中，我们使用了上海交通大学开源的机器人学库，并对该C++库进行C语言接口适配。

六轴机械臂的连杆质量、质心位置及转动惯量，可通过 SolidWorks 自动计算获取。尽管我们已通过人工称重为大部分独立构件赋予实际质量，但受限于均匀密度假设、忽略螺栓螺母等细小构件，自动计算得到的质心位置与转动惯量，仍与实物存在一定偏差；同时，电机厂商提供的力矩常数等参数，也与实际工况不完全一致。因此，若使用自动计算的物理参数，并直接将模型的计算结果作为电机控制指令，控制效果往往不够理想。在实际部署时，需要对这些物理参数进行校准与修正。

2.1.2.1.4 重力补偿（关节角不变时的动力学+传动连杆静力学）实际效果



> 由于撰写此文档时轮足底盘和机械臂正在分离调试中，故此处使用了同构型的另一机械臂的演示视频

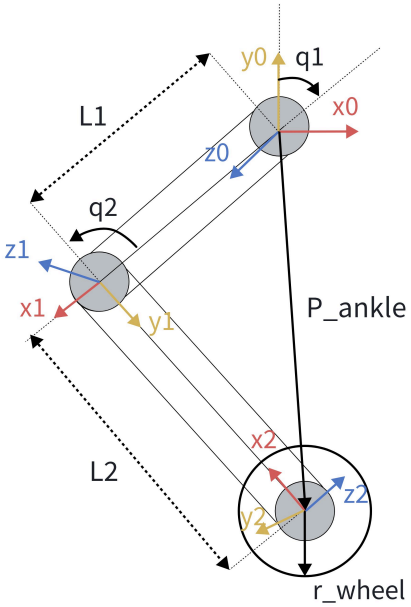
机械臂重力补偿后可以实现任意姿态悬停，极大的提升了控制的效果。

2.1.2.2 底盘

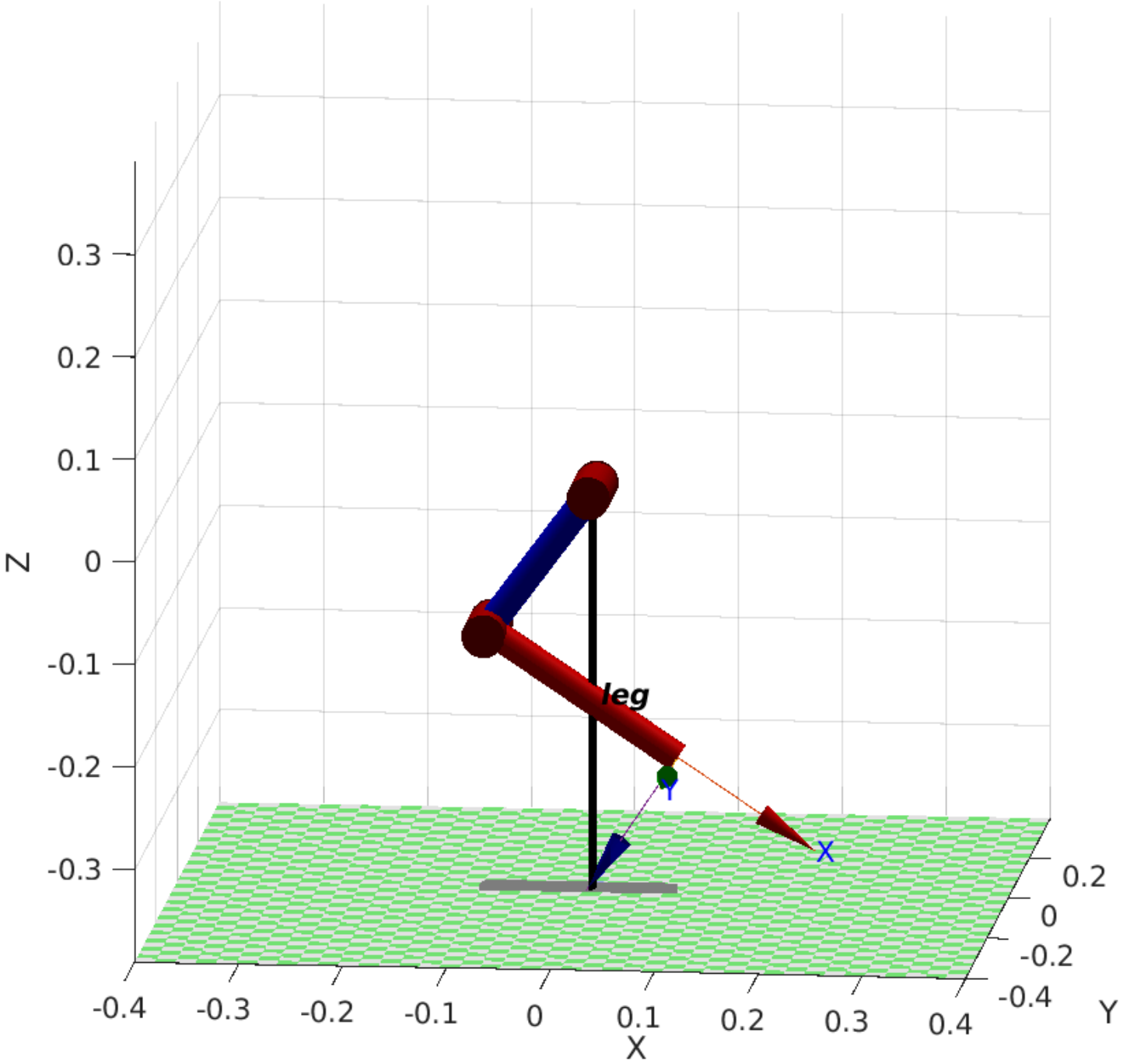
为了实现麦轮轮足底盘的基础运动功能，使用单关节位置速度串级环PID控制+运动学正逆解算+足端力前馈控制。

2.1.2.2.1 连杆系建模

先对单腿坐标系进行建模：



> 坐标系定义



matlab可视化

此处的坐标系{0}称之为腿基坐标系{l}或{leg}，整个机器人有4个腿基坐标系。

2.1.2.2.2 单腿运动学解算

正运动学解算

matlab代码如下：

代码块

```
syms L1 L2 real
% 正运动学
syms q1 q2
```

```
4
5   x_leg = L1*sin(q1) + L2*sin(q1+q2);
6   y_leg = -L1*cos(q1) - L2*cos(q1+q2);
7
8   x_leg_fun=matlabFunction(x_leg);
9   y_leg_fun=matlabFunction(y_leg);
10
11  ccode(x_leg)
12  ccode(y_leg)
```

C代码结果如下：

代码块

```
1   /**
2    * @brief 腿部正运动学计算
3    *
4    * @param q1 关节角1
5    * @param q2 关节角2
6    * @param xyz_leg 轮电机轴心在腿的基座标系下的位置
7    */
8   void Leg_Fkine(const float q1, const float q2, float xyz_leg[3]) {
9       xyz_leg[0] = L2 * fast_sin_f32(q1 + q2) + L1 * fast_sin_f32(q1);
10      xyz_leg[1] = -L2 * fast_cos_f32(q1 + q2) - L1 * fast_cos_f32(q1);
11      xyz_leg[2] = 0.0f;
12  }
```

逆运动学解算

matlab代码如下：

代码块

```
1   syms L1 L2 real
2
3   % 逆运动学
4   syms x_leg y_leg
5
6   d_ = (x_leg*x_leg+y_leg*y_leg)^0.5;
7   alpha_ = acos(x_leg/d_);
8   theta_ = acos((L1*L1 + d_*d_ - L2*L2)/(2*L1*d_));
9   gamma_ = atan2(y_leg+L1*sin(theta_+alpha_),x_leg-L1*cos(theta_+alpha_));
10  q1 = -alpha_ - theta_ + pi/2;
11  q2 = gamma_ + theta_ + alpha_;
12
13  q1_leg_fun=matlabFunction(q1);
```

```
q2_leg_fun=matlabFunction(q2);

ccode(q1)
ccode(q2)
```

C代码结果如下：

代码块

```
/**
 * @brief 腿部逆运动学计算
 *
 * @param xyz_leg 轮电机轴心在腿的基座标系下的位置
 * @param q1 关节角1
 * @param q2 关节角2
 * @return uint8_t 计算成功与否标志，1成功，0失败
 */
uint8_t Leg_Ikine(const float xyz_leg[3], float* q1, float* q2) {
    const float x = xyz_leg[0];
    const float y = xyz_leg[1];
    const float x2 = x * x;
    const float y2 = y * y;
    const float sqrt_x2y2 = sqrtf(x2 + y2);
    const float acos_val1 =
        acosf((1.0f / sqrt_x2y2 * (L1 * L1 - L2 * L2 + x2 + y2)) / (L1 * 2.0f));
    const float acos_val2 = acosf(x * 1.0f / sqrt_x2y2);

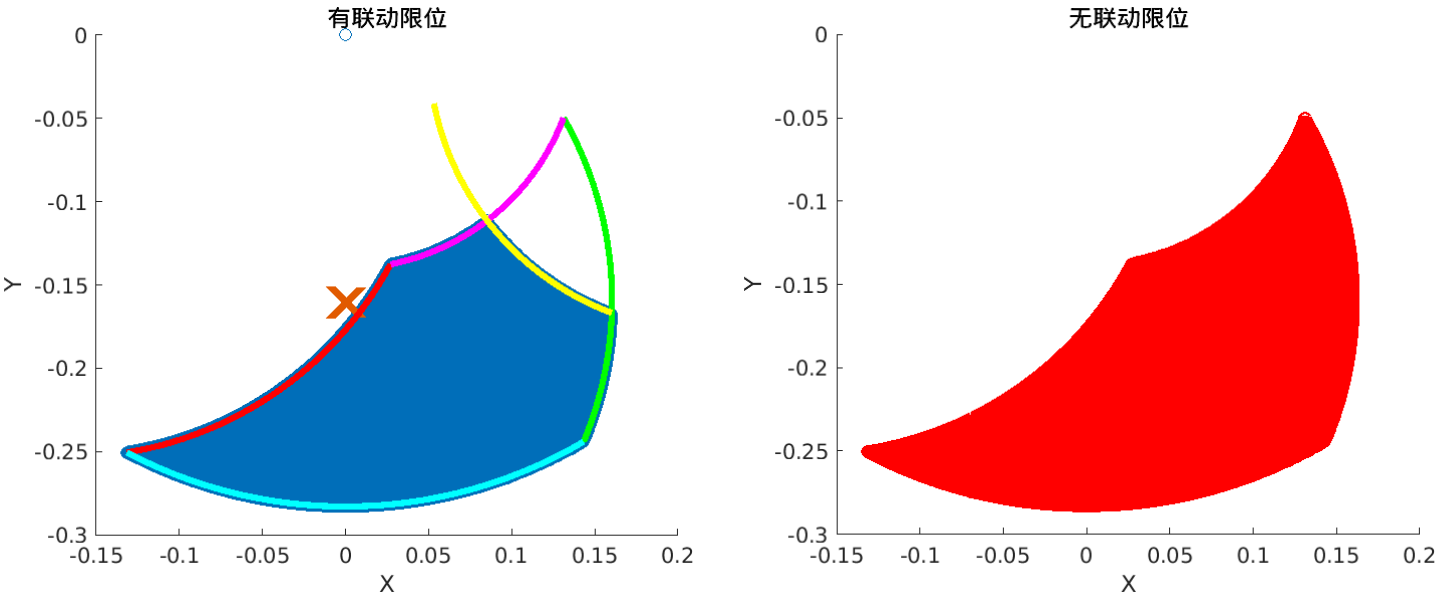
    // clang-format off
    float q1_temp = pi / 2.0f - acos_val1 - acos_val2;
    float q2_temp = atan2f(y + L1 * fast_sin_f32(acos_val1 + acos_val2),
                           x - L1 * fast_cos_f32(acos_val1 + acos_val2))
        + acos_val1 + acos_val2;
    // clang-format on

    // 检验结果
    if (isnan(q1_temp) || isnan(q2_temp)) return 0;

    *q1 = q1_temp;
    *q2 = q2_temp;
    return 1;
}
```

工作空间分析

下面为第一版底盘限位下的工作空间可视化图：



> {l}下轮轴点的工作空间。土黄色‘X’标记为麦轮紧贴关节电机下方时轮轴点的位置

可见在腿长较短时无法很好地向后方伸展，这将导致在上台阶时无法及时调整姿态从而容易向后翻倒。我们在第二版迭代中解决了这个问题。

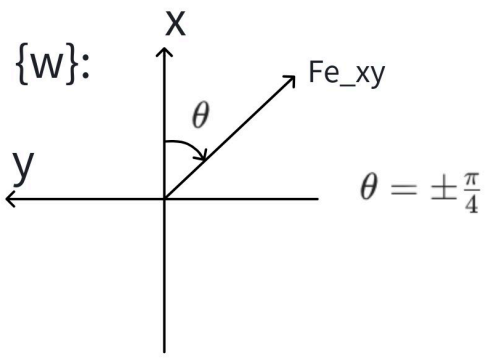
2.1.2.2.3 单腿足端力映射

为了方便后续的控制算法部署，我们首先对单腿足端力到关节力矩和轮轴力矩的映射进行计算。

麦克纳姆轮单刚体动力学计算：

由于轮足与球足相比多出了一个“轮连杆”，因此需要单独对轮进行建模。

首先定义轮坐标系{w}或{wheel}，假定轮轴始终与接触面平行，且接触面始终水平。



> {w}的俯视图，原点位于轮中心，z始终与接触面垂直，x朝车体正前，其中 F_{exy} 表示轮的受力方向， θ 角与该轮的AB型有关

再定义部分物理量：

符号	含义

τ_w	轮电机对轮的力矩，以 y_w 为正方向
F_e	地面对轮的力
F_w	Link2对轮的力
I_w	轮的惯性张量
V_c	轮的质心速度
m_w	轮的质量
ω	轮相对于惯性系的交速度
r_{ce}	质心到触地点的位矢

列出牛顿方程和欧拉方程：

$$m_w\dot{\vec{v}}_c = \vec{F}_e + \vec{F}_w + \vec{G}$$
$$I_c\dot{\vec{\omega}} + \vec{\omega} \times (I_c\vec{\omega}) = \vec{r}_{ce} \times \vec{F}_e + \vec{\tau}_w$$

作单刚体分析，已知： m_w、$\dot{V}_c$、F_e、I_c、$\dot{\omega}$、ω、r_{ce}、G，求解 F_w、I_w：

以上问题经过适当的简化后有如下matlab代码：

代码块

```
% 本节坐标系均为轮坐标系，x向车前，z竖直向上,假设接触面始终水平
syms m_w real %轮质量
syms F_e [3,1] real %地对轮的力
syms vd_c [3,1] real %质心平动速度
syms I_c [3,3] real %轮转动惯量
syms w [3,1] real %角速度
syms wd [3,1] real %角加速度
syms r_ce [3,1] real %质心到触地点的位移矢量
syms r real %轮半径
syms G [3,1] real %重力

% 假设条件
r_ce = [0 0 -r]';
w(1) = 0;
wd(1) = 0;
G = [0 0 -m_w*9.78]';

```

```
F_w = m_w*vd_c - F_e - G;
tau_w = I_c*wd - cross(r_ce,F_e) ;%忽略w x (I_c.*w)

F_w = simplify(F_w);
tau_w = simplify(tau_w);

ccode(F_w)
ccode(tau_w)
```

最终得到C代码如下

代码块

```
/* 参数定义 */
// NOLINTBEGIN
CONSTEXPR float m_w = 0.547f;   // 轮质量 //NOLINT
CONSTEXPR float r = WHEEL_R;  // 轮半径 //NOLINT
CONSTEXPR float I_c[3][3] = {
    // 轮绕质心的转动惯量矩阵
    {0, 0.0f, 0.0f},
    {0.0f, 3.63e-5, 0.0f},
    {0.0f, 0.0f, 0},
};

/**
 * @brief 计算link2对轮的力
 * @note 预设如下条件:
            gravity = [0 0 -9.78]';
 * @param F_e_wheel 地对轮的力
 * @param vd_c_wheel 轮质心加速度
 * @param F_w_wheel link2对轮的力（输出）
 */
void Fw_Calc(const float F_e_wheel[3], const float vd_c_wheel[3], float
F_w_wheel[3]) {
    F_w_wheel[0] = -F_e_wheel[0] + m_w * vd_c_wheel[0];
    F_w_wheel[1] = -F_e_wheel[1] + m_w * vd_c_wheel[1];
    F_w_wheel[2] = -F_e_wheel[2] + m_w * (9.78 + vd_c_wheel[2]);
}

/**
 * @brief 计算link2对轮的力矩输出
 * @note 预设如下条件:
            r_ce = [0 0 -r]';
            w_x = 0;
            wd_x = 0;
 *
```

```
33     * @param F_w_wheel link2对轮的力
34     * @param w_y 轮绕y轴角速度
35     * @param w_z 轮绕z轴角速度
36     * @param wd_y 轮绕y轴角加速度
37     * @param wd_z 轮绕z轴角加速度
38     * @param tau_w link2对轮的力矩（输出）
39     */
40    void Tauw_Calc(const float F_e_wheel[3], const float wd_y, const float wd_z,
      float* tau_w) {
41        tau_w[0] = -F_e_wheel[1] * r + I_c[0][1] * wd_y + I_c[0][2] * wd_z;
42        tau_w[1] = F_e_wheel[0] * r + I_c[1][1] * wd_y + I_c[1][2] * wd_z;
43        tau_w[2] = I_c[2][1] * wd_y + I_c[2][2] * wd_z;
44    }
45    // NOLINTEND
```

至此，可以通过轮的加速度、角加速度、期望足端力计算得到连杆2对轮的力 F_w 与连杆2对轮的力矩 τ_w 。

连杆正静力学解算

为了方便底盘在维修架上的调试，此处将机器人的底盘基座视为固定基，并考虑连杆质量用于重力补偿。

考虑到连杆质量的动力学计算较为复杂，且连杆质量相对于底盘基座质量而言很小，故在正常运动状态将连杆质量忽略，即连杆1和连杆2的重力置零，仅使用虚功原理进行静力学计算。

计算如下，部分物理量已在上文定义过：

符号	含义
q	关节角度， $q = [q1, q2]^T$
j	电机转角， $j = [j1, j2]^T$
A	传动比矩阵， $q = Aj$ ，其中 $A = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$
G_i	连杆i所受重力
r_c	连杆质心位矢， $r_c = [r_{c1}, r_{c2}]^T$
r_w	轮质心位矢
q_w	连杆2末端转角， $q_w = q1 + q2$

在 $\{i\}$ 腿基坐标系下：

- 驱动力：$\boldsymbol{\tau} = [\tau_1, \tau_2]^T$
- 环境力：$[\boldsymbol{G}_1^T, \boldsymbol{G}_2^T, -\boldsymbol{F}_w^T, -\tau_w^T]^T$，记 $\boldsymbol{G} = \begin{bmatrix} \boldsymbol{G}_1 \\ \boldsymbol{G}_2 \end{bmatrix}$

虚功表达式：

- 驱动力虚功：$\delta W_\tau = \boldsymbol{\tau}^T \delta \boldsymbol{j}$
- 重力虚功：$\delta W_G = \boldsymbol{G}^T \delta \boldsymbol{r}_c$
- 轮对 link2 的虚功：$\delta W_w = -\boldsymbol{F}_w^T \delta \boldsymbol{r}_w - \tau_w \delta q_w$（$\tau_w$ 为标量）

虚功平衡方程：

$$\delta W_\tau + \delta W_G + \delta W_w = 0$$

代入展开：

$$\boldsymbol{\tau}^T \boldsymbol{A}^{-T} \delta \boldsymbol{q} + \boldsymbol{G}^T \boldsymbol{J}(\boldsymbol{r}_c, \boldsymbol{q}) \delta \boldsymbol{q} - \boldsymbol{F}_w^T \boldsymbol{J}(\boldsymbol{r}_w, \boldsymbol{q}) \delta \boldsymbol{q} - \tau_w \, d(q_1 + q_2) = 0$$

可整理为：

$$\left[\boldsymbol{\tau}^T \boldsymbol{A}^{-T} + \boldsymbol{G}^T \boldsymbol{J}(\boldsymbol{r}_c, \boldsymbol{q}) - \boldsymbol{F}_w^T \boldsymbol{J}(\boldsymbol{r}_w, \boldsymbol{q}) - [\tau_w, \tau_w] \right] \delta \boldsymbol{q} = 0$$

转化为matlab代码如下：

代码块

```
% 如未标注，本节均在腿基坐标系中计算
syms L1 real %link1长
syms L2 real %link2长
syms q [2,1] real % 关节角
syms F_w [3,1] real %link2对轮的作用力
syms tau_w real %link2对轮的力矩
syms tau [2,1] real %抽象关节力矩
syms G1_ [3,1] real %link1的重力
syms G2_ [3,1] real %link2的重力
syms r_c1_ [3,1] real %link1质心位矢
syms r_c2_ [3,1] real %link2质心位矢
```

```
syms r_w [3,1] real %轮质心位矢
syms r_1_c1_ [3,1] real %link1质心在系1下的位矢
syms r_2_c2_ [3,1] real %link2质心在系2下的位矢
% 由于机体少有侧倾情况，为简化运算，此处指定重力在腿基坐标系z方向上为0
G1_(3) = 0;
G2_(3) = 0;
% 辅助函数，计算绕z轴旋转的旋转矩阵
R_z_fun = @(theta) [    cos(theta)    -sin(theta)    0;
                        sin(theta)    cos(theta)     0;
                        0             0              1];

R_01 = R_z_fun(q(1) - pi/2);

R_12 = R_z_fun(q(2)) * sym([-1     0     0;
                        0      0     1;
                        0      1     0]);

t_01 = [L1*sin(q(1))    -L1*cos(q(1))   0]' ;

t_02 = [L1*sin(q(1)) + L2*sin(q(1)+q(2))    -L1*cos(q(1)) - L2*cos(q(1)+q(2))
 0]';

T_01 = [    R_01    t_01;
            [0 0 0] 1   ];
T_02 = [    R_01*R_12   t_02;
            [0 0 0]     1   ];

temp_c1 = ( T_01*[ r_1_c1_' 1]' );
r_c1_ = temp_c1(1:3);
temp_c2 = ( T_02*[ r_2_c2_' 1]' );
r_c2_ = temp_c2(1:3);
r_w = t_02;
% assume(Fx,'real')
% assume(Fy,'real')
% assume(Fz,'real')

% 传动比，不在此处计算
% A = [1 0;-1 1];

tau = (( -( G1_'*jacobian(r_c1_,q) + G2_'*jacobian(r_c2_,q) ) +
F_w'*jacobian(r_w,q) + [tau_w tau_w] ))';
tau = simplify(tau);
tau_fun = matlabFunction(tau)
assume(tau,'real')
ccode(tau)
```

> 结果较为复杂，此处不做展示

至此，我们可以将目标足端力 F_e 映射为抽象关节力矩与轮轴力矩。

2.1.2.2.4 单刚体简化模型

为什么使用二次规划

笔者在查阅四足机器人运控相关资料时，发现在强化学习方案诞生前，学术界/业界使用较为广泛的力前馈计算算法是MPC算法，这是由于四足机器人是一个高度欠约束系统，在满足牛顿方程与欧拉方程约束的情况下仍然有多余自由度：

$$\dot{P} = F^e$$

> 牛顿方程，系统动量对时间的导数等于其所受的合外力

$$\dot{L}_c = I_c\dot{\omega} + \omega \times (I_c\omega)$$

$$I_c\dot{\omega} + \omega \times (I_c\omega) = \sum_k M_c(F_k^{(e)})$$

> 欧拉方程，质点系相对于质心的动量矩对时间的导数等于所有外力对质心的主矩

例如，在满足上述方程组（3+3=6 个方程）的前提下，12 自由度的四轮足麦轮机器人仍存在 6 个冗余自由度，因此通常需要采用优化方法求解最优关节力矩。MPC 的核心之一正是通过优化方法（通常为二次规划）得到使代价函数最小化的控制量序列。在此过程中，我们可以方便地为二次规划器添加等式或不等式约束，从而实现与物理约束（如足端摩擦锥约束）的结合；此外，MPC 是基于模型的控制算法，能够更好地发挥先验模型的作用。

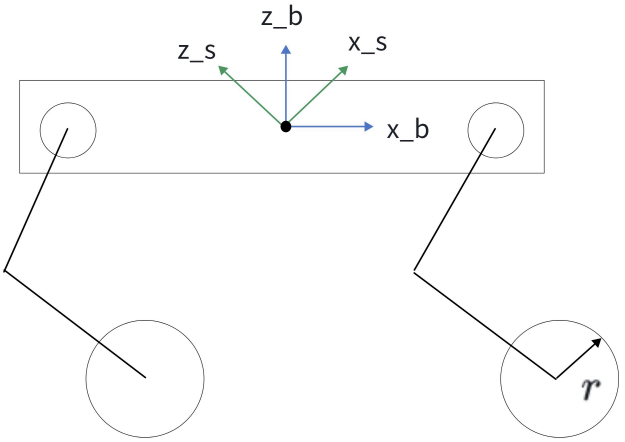
综合种种优点，我们认为MPC是较好的前馈计算算法。但是鉴于队伍先前MPC调试经验的缺乏以及4足构型本身具有自稳定性，截至我们撰写时，我们暂时计划先仅使用二次规划进行前馈计算，待调试稳定后再实现MPC算法。

由于腿部连杆是一个非线性极强的系统，使用二次优化时需要进行额外处理，且实现考虑腿部连杆质量的动力学计算较为复杂，在参考了宇树的《四足机器人控制算法——建模、控制与实践》书中内容后，我们决定将整个机器人的多刚体系统简化为单刚体处理。其中自变量设计为4腿的足端力，每个力有3个维度，共12自由度。

单刚体动力学方程

定义机体坐标系{b}或{body}，原点位于机器人质心处，x_b朝正前，z_b朝上；

世界坐标系{s}，原点与{b}原点重合，z_s竖直向上，且有x_s、z_s、x_b共面，即x_s始终指向机器人前进方向。



> 坐标系定义示意图

首先根据刚体动力学方程定义如下量：

$$A = \begin{bmatrix} I_{3\times3} & I_{3\times3} & I_{3\times3} & I_{3\times3} \\ p_{e0}^{\times} & p_{e1}^{\times} & p_{e2}^{\times} & p_{e3}^{\times} \end{bmatrix}_{6\times12}$$

> p_{ei} 为第i个轮足端点在世界系下的位矢，右上角标x代表3维向量的叉乘操作符，即 $p_{ei}^{\times}$ 为3x3的叉乘矩阵

$$b = \begin{bmatrix} m(\dot{v}_s - g) \\ R_{sb} I_b R'_{sb} \dot{\omega}_s \end{bmatrix}$$

> R_{sb} 为{s}下{b}的旋转矩阵， I_b 为{b}下机器人的惯量矩阵， $\dot{v}_s$ 与 $\dot{\omega}_s$ 为机器人在{s}下的加速度和角加速度

$$f = [f_{0s}, f_{1s}, f_{2s}, f_{3s}]^T$$

> f_{is} 为地面对第i个足端触地点的力在{s}下的表达

根据牛顿方程与忽略小项 $\omega \times (I_c \omega)$ 后的欧拉方程，单刚体满足：

$$Af = b$$

选择代价函数和约束

对于代价函数J，我们考虑如下因素

- 希望足端力尽可能满足动力学方程的约束
- 希望足端力尽可能小

- 希望足端力不要突变（限制急动度）

于是将J设计如下：

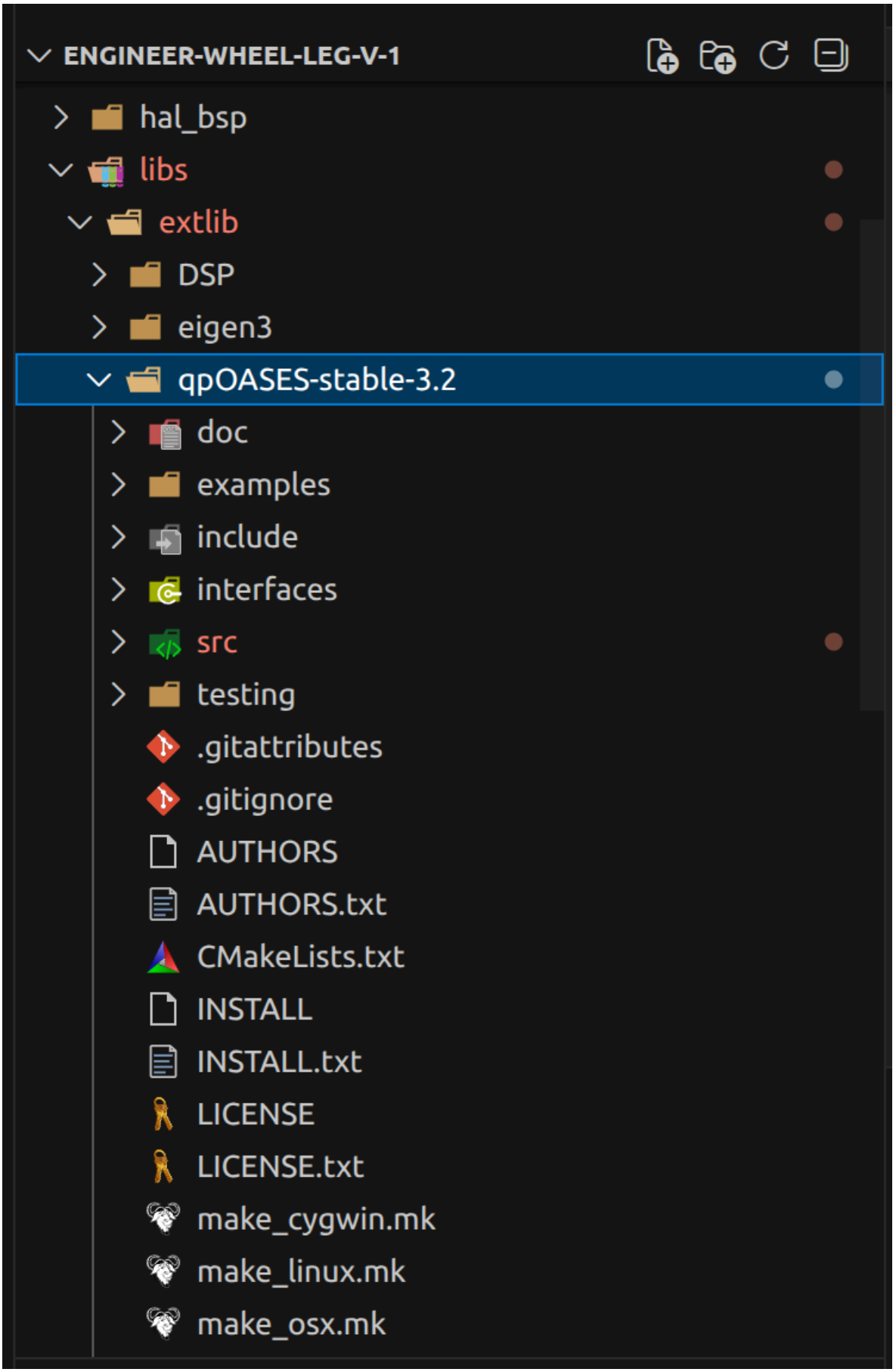
$$J = (Af - b)'S(Af - b) + f'Wf + (f - f_{last})'U(f - f_{last})$$

> S,W,U为权重矩阵， f_{last} 为上一个时间步计算得到的足端力

此外，还需要添加物理约束

- 麦克纳姆轮的受摩擦力方向沿辊子轴向，而垂直方向近似不受力
- 麦克纳姆轮与地面的摩擦因数有限，正压力有限时摩擦力有最大值

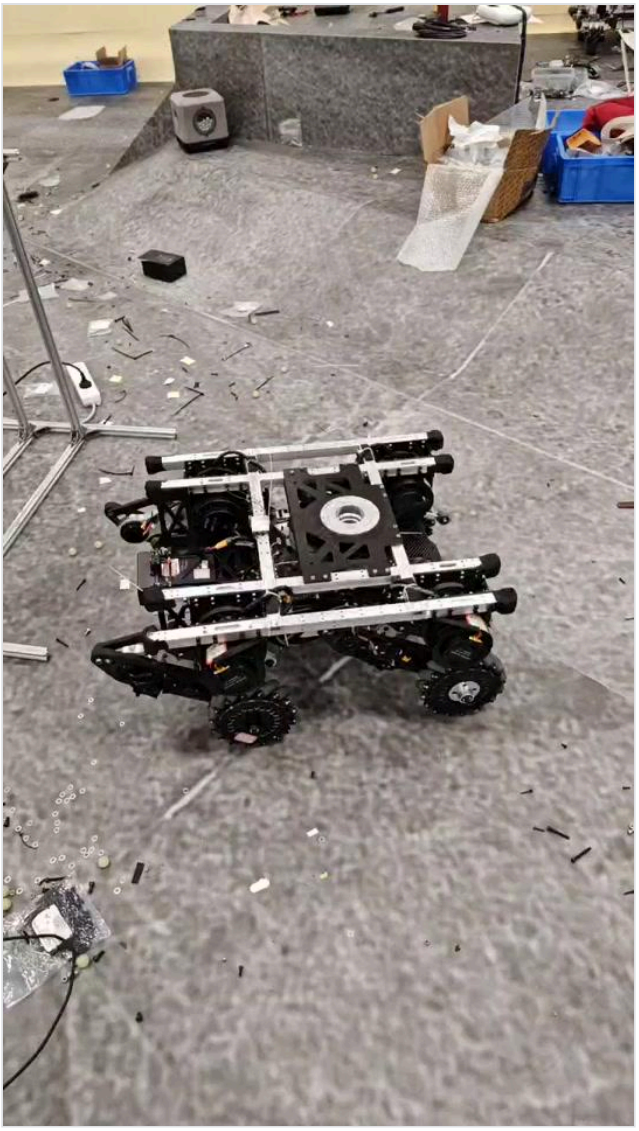
在代码中，我们准备使用c++二次规划库qpOASES求解：



> qpOASES的目录

2.1.2.2.5 基本运动效果演示

由于调试时间比较紧张，在我们撰写时还未实现二次规划前馈计算，下面的展示视频基于简化的前馈计算方法：



2.1.3 进阶控制流程

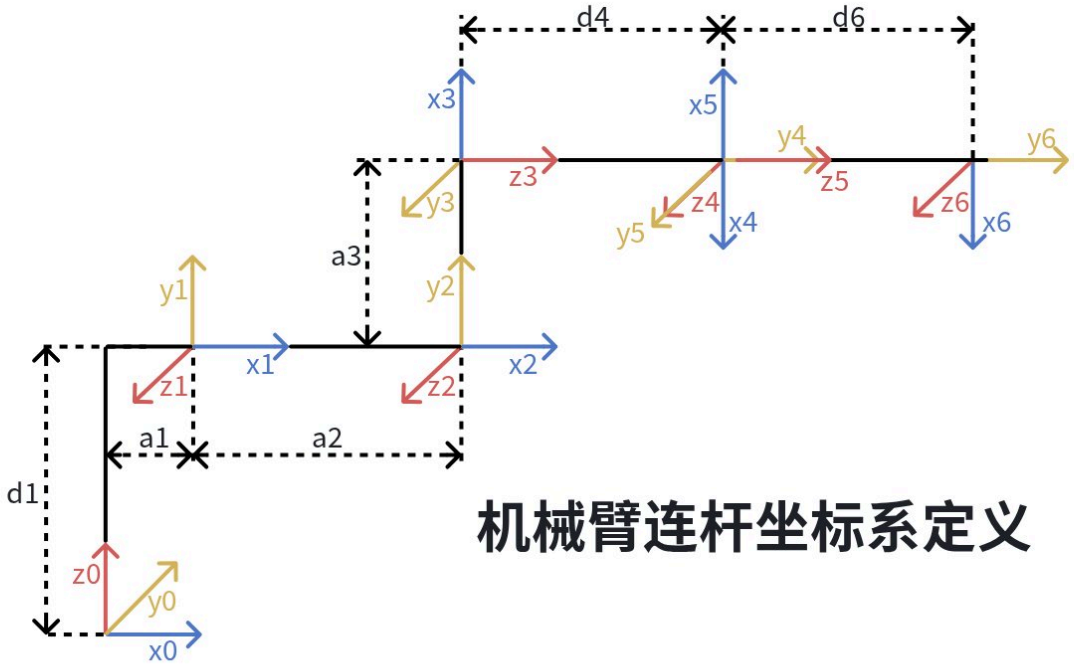
2.1.3.1 机械臂

2.1.3.1.1 运动学逆解算

为了实现异构遥操作控制器的控制以及后续的视觉方案，需要实现由末端齐次变换矩阵到关节角度的映射。虽然可以通过迭代法（数值法）进行求解，但这种方法往往比较费时且当目标位姿与初始迭代位姿误差较大时容易无解，另外还存在解臂型不稳定的问题。总之，数值法对于需要稳定有解且可重复定位的场景而言无法完全满足需求。

因此，我们在制作机械臂时有意使机械臂构型满足pieper准则，即末端3轴线交于一点，保证工作空间内一定有解析解，并求解解析解（封闭解）。

回顾一下连杆系的结构：

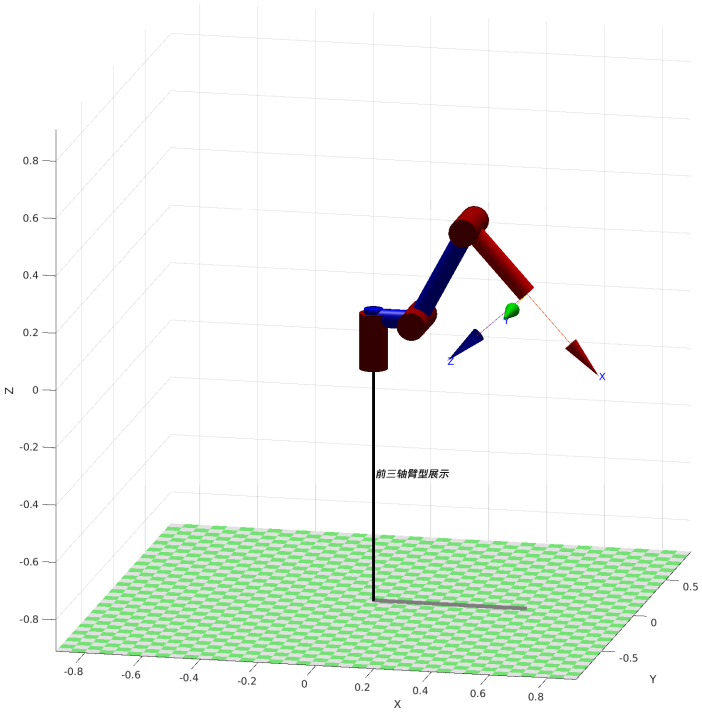


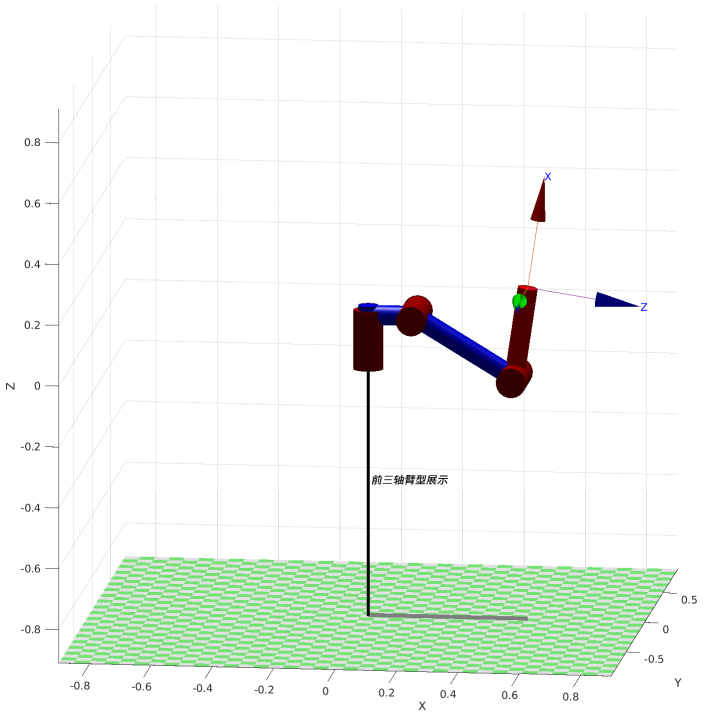
连杆坐标系定义

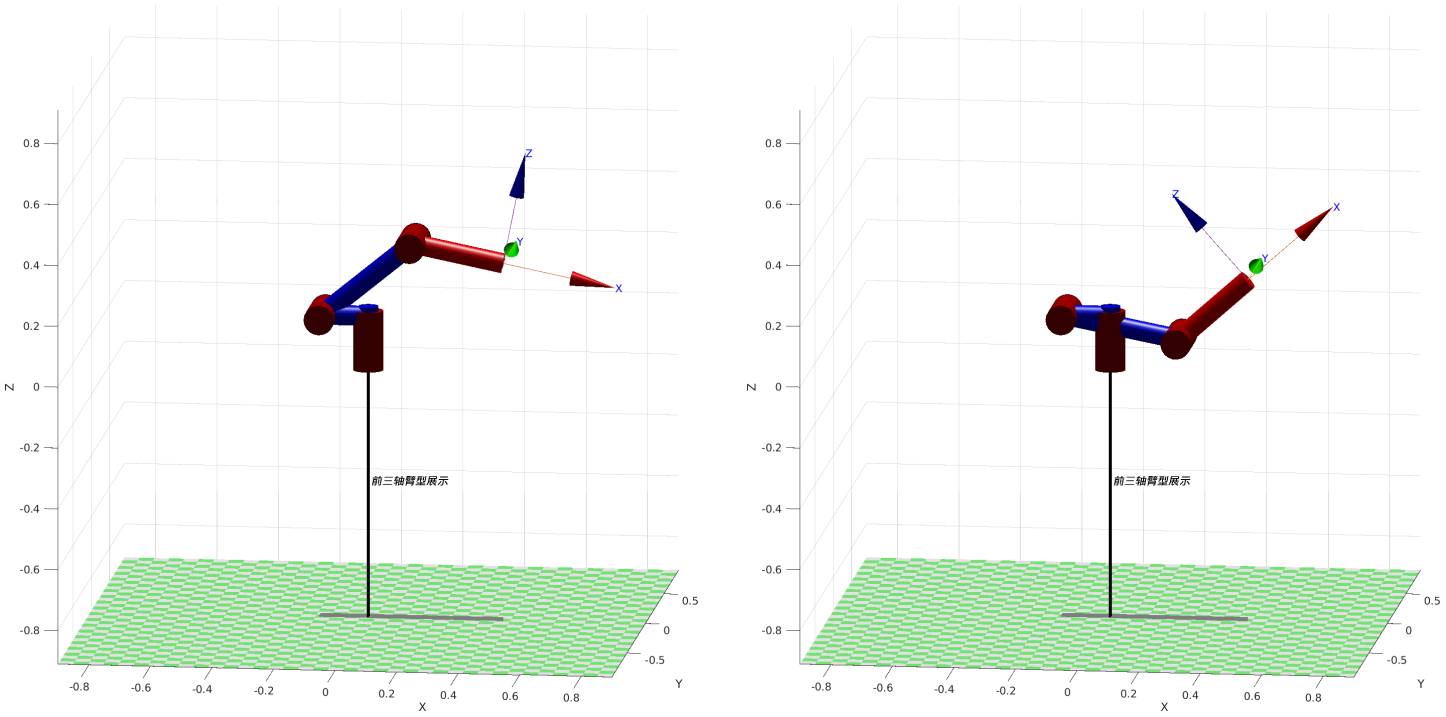
可以发现，末三轴的交点具有特殊性，我们称之为“腕点”**W**。

一旦 T_6^0 确定，由于腕点总在 O_6 沿 y_6 负向d6处,故腕点的位置就随之确定。同时，腕点的位置是{4}的原点，而{4}的原点位置只与前三个关节角有关而与后三个关节角无关。在非奇异条件下，我们可以得出此时前三个关节角的解一定是有限个。

事实上，对于依次为yaw-pitch-pitch-roll-pitch-roll的关节构型而言，在工作空间内，当三轴臂末端端点（即六轴臂的腕点）确定时，前三轴的“臂型”一共有四种:







> 前三轴臂型展示，依次为正对高臂、正对低臂、反对高臂、反对低臂

由于实际的角度限位，最终需要考虑的只有正对高臂、反对低臂。

matlab代码如下：

代码块

```
% 腕点逆解算
syms xw yw zw real % 定义腕点位置符号变量（实数）：xw, yw, zw

% 说明：a(i), d(i) 来自 DH 参数（连杆长度/偏距）


% j1正对高臂
l2=a(2);% 第2连杆等效长度
l3=(a(3)^2 + d(4)^2)^0.5;% 第3段等效长度（由 a3 与 d4 合成）

% ---- 关节1角度 ----
theta1=atan2(yw,xw);% 基座旋转角，朝向腕点在XY平面的方位角

lwr=(xw^2 + yw^2)^0.5;% 腕点到基座Z轴的水平距离（r）
lwz=zw;% 腕点Z坐标
l1w=((lwr - a(1))^2 + (lwz - d(1))^2)^0.5;% 从关节2原点到腕点的直线距离
% ---- 关节2与关节3角度（余弦定理）----
alpha1=atan2(zw - d(1), lwr - a(1));
theta2=alpha1 + acos( (l2^2 + l1w^2 - l3^2) / (2*l2*l1w) );
theta3=-(pi - acos( (l2^2 + l3^2 - l1w^2) / (2*l2*l3) ));

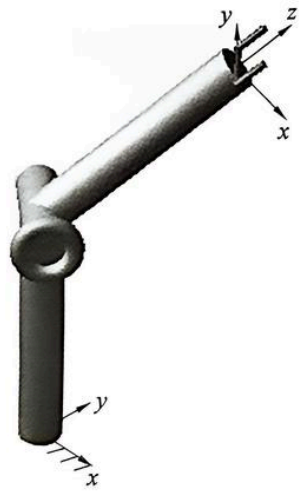
q1_fun=matlabFunction(theta1);% q1(xw,yw,zw)
q2_fun=matlabFunction(theta2);% q2(xw,yw,zw)
q3_fun=matlabFunction(theta3 - atan(a(3)/d(4)));% q3补偿a3带来的偏角
```

```
25
26
27  % j1反对低臂
28  l2=a(2);% 第2连杆等效长度
29  l3=(a(3)^2 + d(4)^2)^0.5;% 第3段等效长度（由 a3 与 d4 合成）
30  % ---- 关节1角度：与分支1相差 ----
31  theta1=atan2(yw,xw) + pi;% 基座旋转到反向
32
33  lwr=(xw^2 + yw^2)^0.5;% 腕点到基座Z轴的水平距离（r）
34  lwz=zw;% 腕点Z坐标
35  % 注意这里使用 -lwr - a(1)，表示采用反向几何构型
36  l1w=((-lwr - a(1))^2 + (lwz - d(1))^2)^0.5;% 从关节2原点到腕点的直线距离
37  % ---- 关节2与关节3角度（余弦定理）----
38  alpha1=atan2(zw - d(1), -lwr - a(1));
39  theta2=alpha1 + acos( (l2^2 + l1w^2 - l3^2) / (2*l2*l1w) );
40  theta3=-(pi - acos( (l2^2 + l3^2 - l1w^2) / (2*l2*l3) ));
41
42  q1_fun2=matlabFunction(theta1);% q1(xw,yw,zw)
43  q2_fun2=matlabFunction(theta2);% q2(xw,yw,zw)
44  q3_fun2=matlabFunction(theta3 - atan(a(3)/d(4)));% q3补偿a3带来的偏角
```

当确定了前三轴的关节角度后， T_3^0 也就可以确定，同时已知 T_6^0 ，由于齐次变换矩阵的可逆性，可以得到 $T_6^3 = (T_3^0)^{-1}T_6^0$ 。此时，问题转化为求解三轴共点机械臂即“球腕机械臂”的关节角。

由于最初确定腕点位置时已经考虑了球腕的末端位置，因此 T_6^3 自然满足球腕杆长（d4、d6）的约束，故接下来只需要使关节角4、5、6满足 R_6^3 的约束即可。

在非奇异情况下，球腕姿态逆解算有两组解：



(a)无翻转腕(50°,−130°,80°)



(b)翻转腕(−130°,130°,−100°)

> 来自徐文福教授的《机器人学：基础理论与应用实践》

以下是matlab计算代码：

代码块

```
% 球腕逆解算
syms nx ny nz ox oy oz ax ay az real
R36=[   nx ox ax;
        ny oy ay;
        nz oz az];

% 第一组解
q5_1=acos(-oz)-theta_offset(5);
q4_1=atan2( oy*sin(q5_1+theta_offset(5)) , ox*sin(q5_1+theta_offset(5)) )-theta_offset(4);
q6_1=atan2( az*sin(q5_1+theta_offset(5)) , nz*sin(q5_1+theta_offset(5)) )-theta_offset(6);
q4_fun1=matlabFunction(q4_1);
q5_fun1=matlabFunction(q5_1);
q6_fun1=matlabFunction(q6_1);

% 第二组解
q5_2=-acos(-oz)-theta_offset(5);
q4_2=atan2( oy*sin(q5_2+theta_offset(5)) , ox*sin(q5_2+theta_offset(5)) )-theta_offset(4);
q6_2=atan2( az*sin(q5_2+theta_offset(5)) , nz*sin(q5_2+theta_offset(5)) )-theta_offset(6);
q4_fun2=matlabFunction(q4_2);
q5_fun2=matlabFunction(q5_2);
q6_fun2=matlabFunction(q6_2);
```

> theta_offset是指定机械臂零点姿态时各关节在标准DH中的theta

至此，我们得到了两组关节1、2、3的解与两组关节4、5、6的解，两两组合后一共有4组解，在实际选取解时只需要选择与当前机械臂关节角误差最小的一组即可。

2.1.3.1.2 运动学奇异点处理

在当前yaw-pitch-pitch-roll-pitch-roll的关节构型下，考虑实际关节限位，主要存在2个奇异点，在使用笛卡尔空间控制时（比如遥操作控制器控制）需要分别处理。

1. 前三轴位置奇异

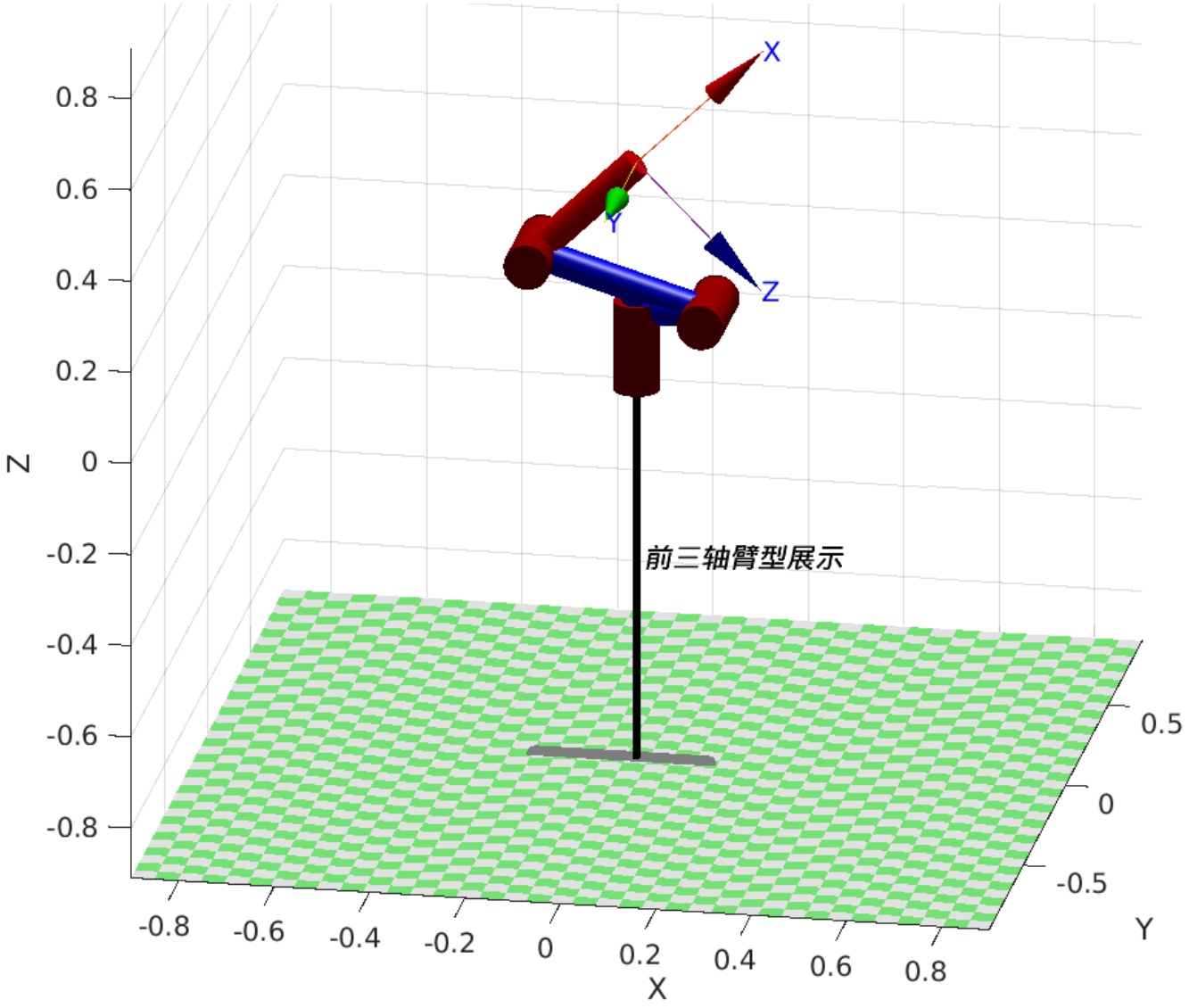
   当腕点位于关节1的轴线处时，机械臂将失去关节1的自由度。

用方程式表达如下

$$\frac{\partial P_w^0}{\partial q_1} = 0$$

> P_w^0 为腕点在{0}下的坐标， q_1 为关节1的关节角

建模示意如下



> $P_{wx}^0 = P_{wy}^0 = 0$ 时的臂型

尽管机械臂不会长时间停留在该位置，但在经过该位置附近区域时， $\frac{dq_1}{P_{wx}}$ ， $\frac{dq_1}{P_{wy}}$ 会急剧增大。也就是说，当 P_{wx} 、 P_{wy} 发生微小变化时， q_1 会产生剧烈变动，从而引发机械臂快速甩动，对机械结构造成损伤，同时也不利于稳定控制。由于实际连杆长度满足 a2≈d4，机械臂在折叠状态下腕点距离关节 1 转轴很近，因此在常规伸展的工作状态下很难进入该奇异点。所以，我们只需对笛卡尔空间控制的工作空间进行简单限制即可。

```
代码块
1  // 机器人工作空间
2  #define ROBOT_X_MAX 0.65
3  #define ROBOT_X_MIN 0.143
```

2. 后三轴姿态奇异

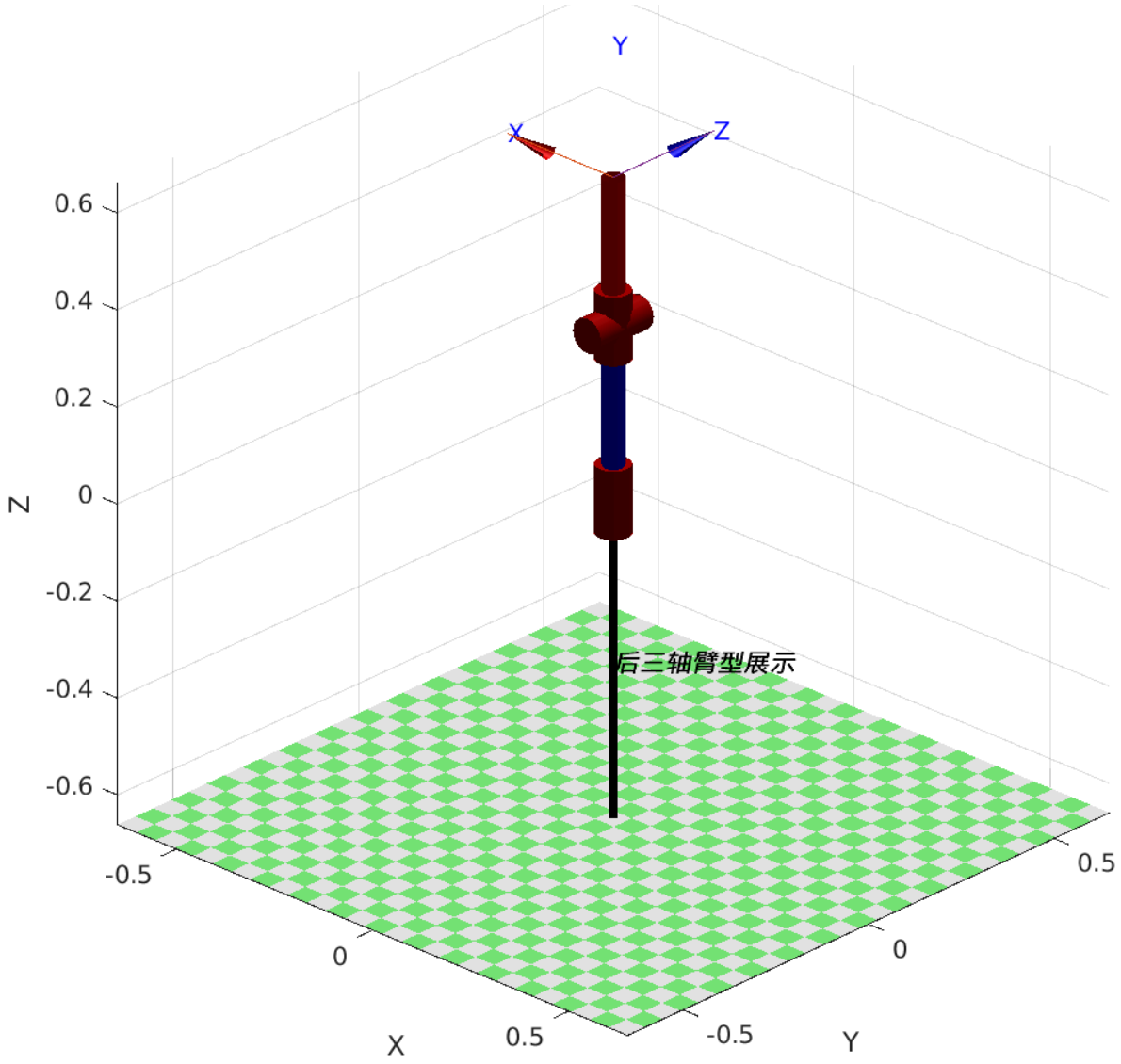
当关节4、6轴线共线时，关节4、6的运动对末端姿态的效果“相同”，关节4、6总共只能提供一个自由度，损失了一个末端姿态自由度。

用方程式表达如下：

$$q_4 \pm q_6 = c(R_6^3)$$

> $c(R_6^3)$ 是一个关于末三轴球腕机械臂姿态的函数，此时 q_4, q_6 只需要满足这一个约束即可，也即 q_4, q_6 有无穷解

建模示意如下：



同样，在此姿态附近，q4、q6 对 R_6^3 的导数陡增，直接的表现为关节速度激增，因此我们采用保持原值平稳过渡到奇异点区域外的策略，能很好的避免上述问题出现。

此外机械臂在腕部奇异点附近工作时，还存在多解切换的需求以避免关节4和关节6的180度翻转，为妥善处理机械臂运动学逆解的切换时产生的角度跳变的问题，我们使用td跟踪微分器（下文有详细介绍）进行了加速度限制，保证了最终关节角度的精度的同时很好地控制了关节的速度变化。

代码块

```
// ========== 第一组解 ==========
float q5_1 = acosf(oz_limited) - theta_offset_5;
float sin_q5_offset_1 = sinf(q5_1 + theta_offset_5);
float q4_1, q6_1;
// 检查是否接近奇异点（sin(q5+offset) ≈ 0）
if (fabsf(sin_q5_offset_1) < 1e-6f) {
    // 奇异点处理：当 q5 接近 0 或 ±π 时，q4 和 q6 不唯一
    q4_1 = last_q->q[3];  // 使用上次的 q4
    q6_1 = last_q->q[5];  // 使用上次的 q6
} else {
    q4_1 = atan2f(oy * sin_q5_offset_1, ox * sin_q5_offset_1) - theta_offset_4;
    q6_1 = atan2f(az * sin_q5_offset_1, nz * sin_q5_offset_1) - theta_offset_6;
}
```

> 仅截取部分代码作实例

2.1.3.1.3 三次多项式轨迹插值

由于机械臂自身的惯量较大，使用pid控制时若阶跃过大很容易引起超调，对固定动作乃至视觉定位的执行不利，因此我们使用了三次多项式插值的方法进行轨迹平滑，并且实现所有关节的同步运动。

(三次多项式)

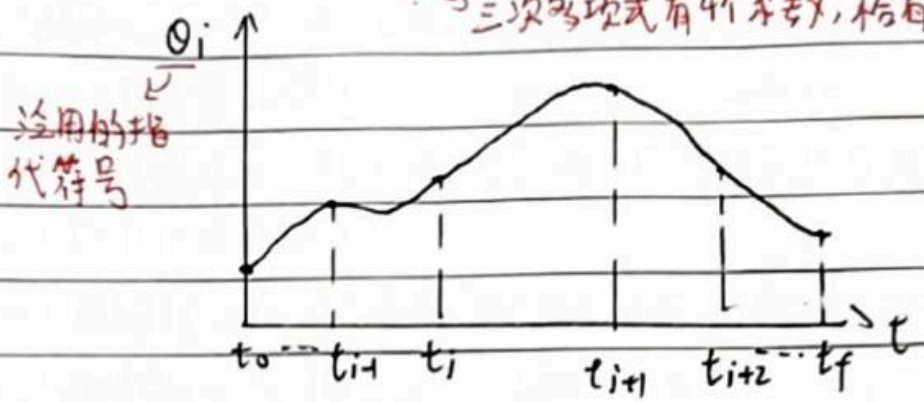
Cubic Polynomials：平滑轨迹规划方法

原则：

轨迹：不同轨迹区段 $[t_i, t_{i+1}]$ 以不同参数的三次函数规划

平滑：定义各函数的边界条件：$\theta(t_i)$, $\theta(t_{i+1})$, $\dot{\theta}(t_1)$, $\dot{\theta}(t_{i+1})$，包含位置与速度，共4个条件

三次多项式有4个系数，恰有可能求解



求解：

1. 通式：$\theta(\hat{t}) = a_0 + a_1\hat{t} + a_2\hat{t}^2 + a_3\hat{t}^3$

2. 对每个区段：$t \in [t_i, t_{i+1}]$

$\hat{t} = t - t_i$，$t_{i+1} - t_i = \Delta t > 0$

边界条件：

$$\begin{cases} \theta(\hat{t}|_{t=t_i}) = \theta_i = a_0 \\ \theta(\hat{t}|_{t=t_{i+1}}) = \theta_{i+1} = a_0 + a_1\Delta t + a_2\Delta t^2 + a_3\Delta t^3 \\ \dot{\theta}(\hat{t}|_{t=t_i}) = \dot{\theta}_i = a_1 \\ \dot{\theta}(\hat{t}|_{t=t_{i+1}}) = \dot{\theta}_{i+1} = a_1 + 2a_2\Delta t + 3a_3\Delta t^2 \end{cases}$$

联立可解.

写成矩阵：

$$\begin{bmatrix} \theta_i \\ \theta_{i+1} \\ \dot{\theta}_i \\ \dot{\theta}_{i+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & \Delta t & \Delta t^2 & \Delta t^3 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 2\Delta t & 3\Delta t^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

$$\Theta = T_{t+1}(\Delta t) \cdot A$$

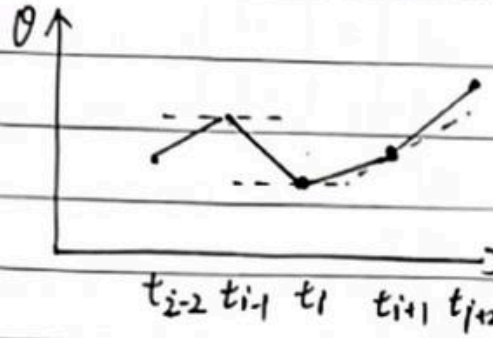
$|T(\Delta t)| = -\Delta t^4 \neq 0$，故恒有解

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -\frac{3}{\Delta t^2} & \frac{3}{\Delta t^2} & -\frac{2}{\Delta t^2} & -\frac{1}{\Delta t} \\ \frac{2}{\Delta t^3} & -\frac{2}{\Delta t^2} & \frac{1}{\Delta t^2} & \frac{1}{\Delta t^2} \end{bmatrix} \begin{bmatrix} \theta_i \\ \theta_{i+1} \\ \dot{\theta}_i \\ \dot{\theta}_{i+1} \end{bmatrix}$$

选择速度条件 $\dot{\theta}_i$ 与 $\dot{\theta}_{i+1}$

1、直接定义：仅在有需求时使用

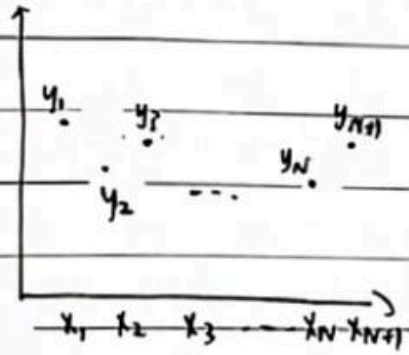
2. 自动生成：如 若 $\dot{\theta}_i$ 在 t_i 前后变号，令 $\dot{\theta}_i = 0$　若 $\dot{\theta}_i$ 在 t_i 前后同号，令 $\dot{\theta}_i = \frac{\frac{\theta_{i+1}-\theta_i}{t_{i+1}-t_i} + \frac{\theta_i-\theta_{i-1}}{t_i-t_{i-1}}}{2}$



不同区段分别插补

3、使过程中加速度连续　不同区段需整体求解

3'. 一般情况（三次样条）



N+1 个点.

x_1 为起点.

$x_2 \sim x_N$ 为过程点

x_{N+1} 为终点.

共有 N 条曲线需计算

记为：$S_j(x) = a_j + b_j x + c_j x^2 + d_j x^3$

$x_j \le x \le x_{j+1}$，$j = 1, \cdots, N$

共 4N 个未知量

对于每个 $S_j(x)$，左右都有2个位置条件 —— 2N

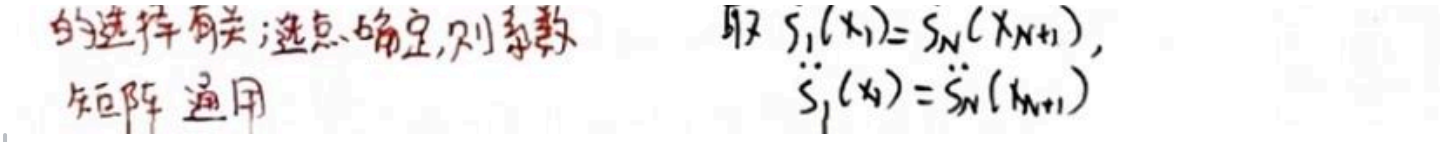
对于每个过程点（共N-1个），都有1个速度、1个加速度条件 —— 2(N-1)

最后2个条件的设定：① $\ddot{S}_1(x_1) = \ddot{S}_N(x_{N+1}) = 0$　定义加速度

② $\dot{S}_1(x_1) = U$，$\dot{S}_N(x_{N+1}) = V$　定义速度

③ 在 $S_1(x_1) = S_N(x_{N+1})$ 时（周期性运动），

注意，最终系数矩阵对角占优.



> 三次多项式插值法的推导

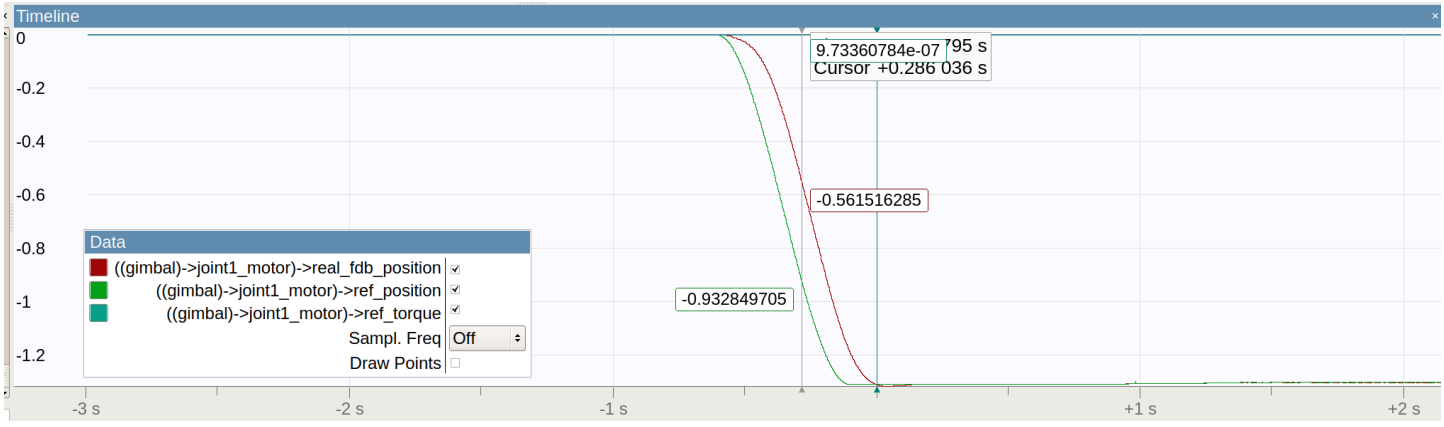
在代码中，我们实现了自编库：

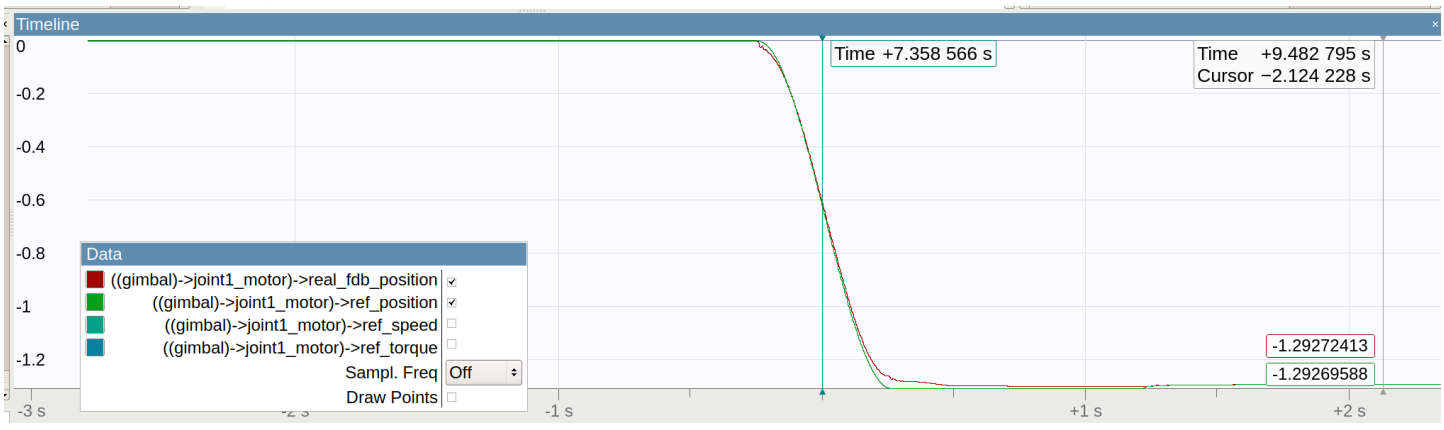
- 插值器单元 `move_plan.c/.h`：将一组轨迹通过点（via_point，包含位置和速度信息）、通过时间（duration）进行分段插值
- 插值器组 `plan_group.c/.h`：将多个插值器单元整合，使用同一组通过时间和各自不同组通过点实现各单元同步运行，同时实现通过点速度的自动计算。亦称为“动作组”

在实际运用时有两种使用方法：

1. 关节空间规划：给定每个关节的通过点与通过时间，对每个关节角单独插值。这种方法适合简单、无避障需求的轨迹规划，优点在于计算量小、单关节速度与加速度更加合理。
2. 笛卡尔空间规划：给定末端位姿（3平移坐标与3欧拉角）通过点，对每个位姿维度单独插值，并在每个时间步进行逆运动学解算。这种方法适合有笛卡尔空间约束的任务（比如将能量单元插入装配柱），缺点是计算量大，并且在运动学奇异点附近关节速度与加速度陡增，需要做额外的奇异处理。

由于平滑轨迹可以离线计算速度与加速度，故配合RNE算法，我们可以计算动力学前馈，实现更加紧密的轨迹跟踪。





> 关节1（yaw），$\frac{2\pi}{3}$（60°），0.5s。上图为无速度、加速度前馈的跟踪效果，下图为有速度、加速度前馈的跟踪效果

接口展示如下：

- `move_plan.h`

代码块

```
//运动规划结构体
typedef struct Move_plan_t {
    Move_plan_status status;
    Move_plan_config* config;
    uint8_t now_line_id; // 当前线段ID
    basic_data_t start_time; // ms
    basic_data_t now_line_start_time; // 当前线段开始时间
   basic_data_t now_line_delta_time; // 当前线段内的时间
    float current_pos;
    float current_vel;
    float current_acc;
} Move_plan;
// 初始化函数
void Move_Plan_Init(void);
Move_plan_config* Move_Plan_Config_Create(void);
void Add_Via_Point(Move_plan_config *config, float pos, float vel);
void Add_Via_Line(Move_plan_config *config, float duration_s);
Move_plan *Move_Plan_Create(Move_plan_config *config);
void Move_Plan_Recalc(Move_plan_config *config);
//执行函数
void Reset_Move_Plan(Move_plan *plan);
void Run_Move_Plan(Move_plan *plan);
//更新函数
void Move_Plan_Update(void);

// 临时规划
Move_plan * Move_Plan_Temp_Create(float start_pos, float start_vel, float
end_pos, float end_vel, float duration_s);

// 查询函数
uint8_t get_Rest_Temp_Plan_cnt(void);
```

- `plan_group.h`

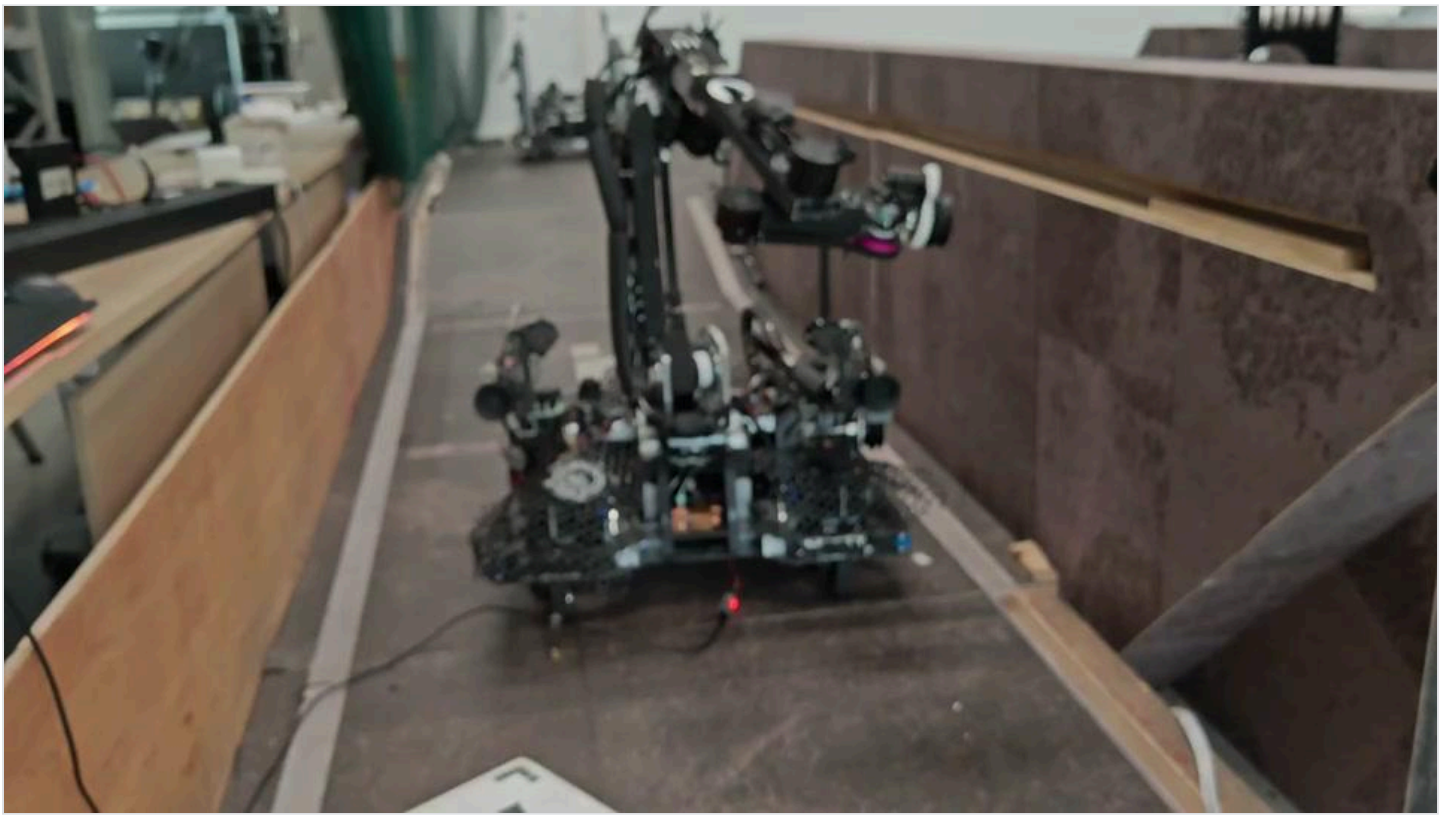
代码块

```
typedef Move_plan** PlanGroup;

PlanGroup Plan_group_Create_Auto(uint8_t plan_cnt, float start_pos[plan_cnt],
                                 uint8_t via_pos_cnt,
                                 float via_pos[plan_cnt][via_pos_cnt],
                                 float duration_s[via_pos_cnt]);
void Plan_group_Temp_Create(PlanGroup temp_group, uint8_t plan_cnt, float start_pos[plan_cnt],
                                 float end_pos[plan_cnt], float duration_s);

void Plan_group_Run(PlanGroup group, uint8_t plan_cnt);
void Plan_group_Reset(PlanGroup group, uint8_t plan_cnt);
```

实际效果如下：



> 25赛季的舵轮机器人的测试轨迹效果

2.1.3.1.4 TD跟踪微分器

尽管三次多项式插值可以应对大部分离线计算的轨迹，但其对于在线计算（比如遥控器控制、遥操作控制器控制）则无能为力。若输入参考信号出现了阶跃（丢包、逆解失败等等），关节仍然会出现超调乃至震荡。为此，我们使用TD跟踪微分器解决这一问题。

TD跟踪微分器主要基于输入信号的二阶导数的绝对值上界进行信号平滑，优点在于其参数速度因子**r**具有加速度的物理意义，易于调试。

其代码实现如下：

代码块

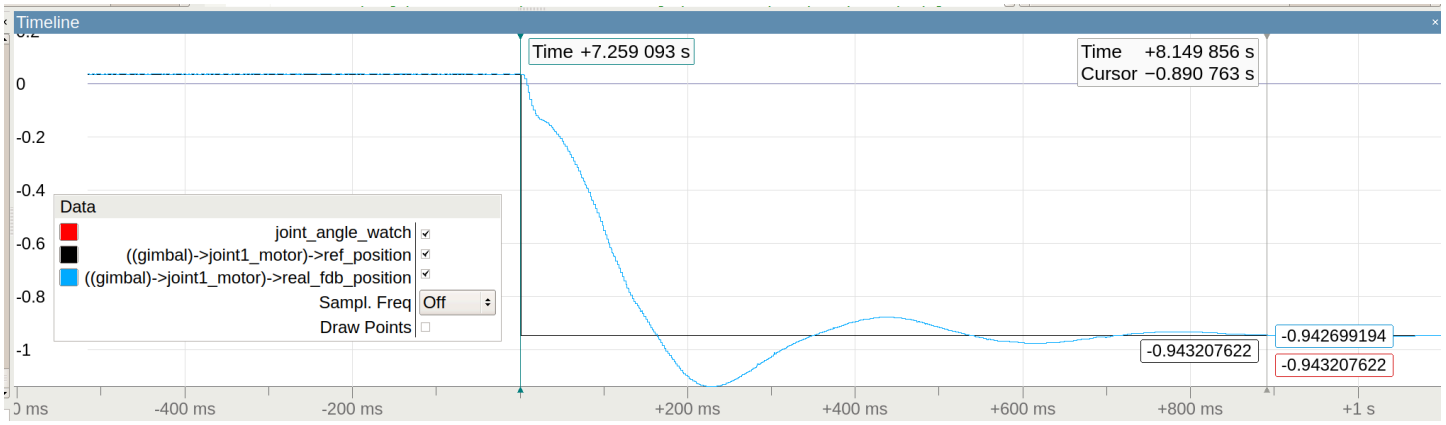
```
//跟踪微分器的参数
typedef struct TD_Config_t {
    float r;//速度因子，r越大，则跟踪微分器对指令跟踪的越快（相位滞后越小，幅值衰减越少），微分出来的值滞后也越小，即越接近真实值，但也会放大噪声
    float h;//步长，一般取系统运算周期即可
    float h0;//滤波因子，一般取h的两倍
} TD_Config;

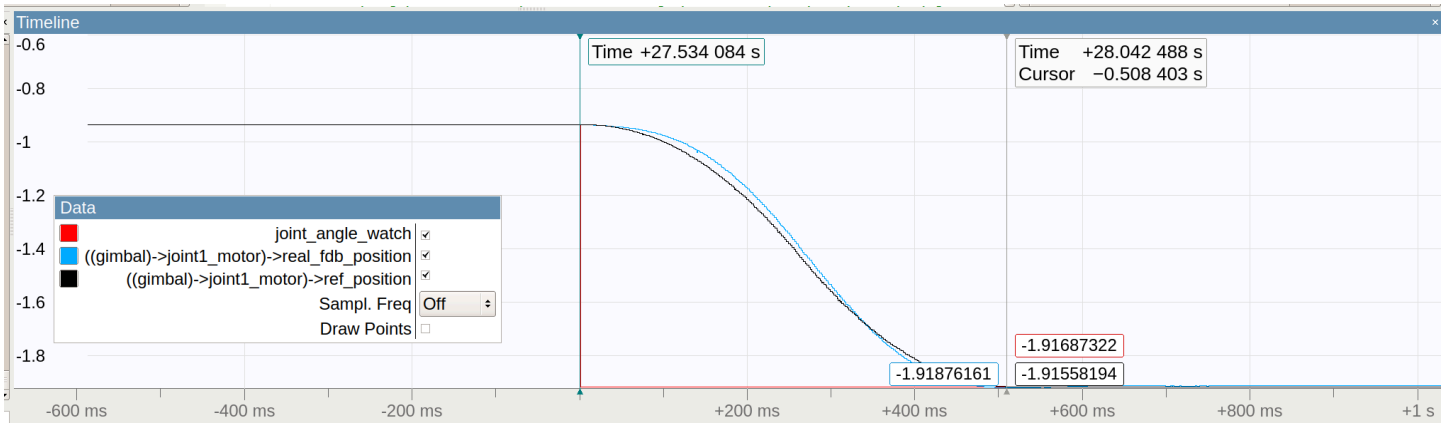
//跟踪微分器
typedef struct TD_t {
    TD_Config config;
    float ref;//目标跟踪信号
    float v1;//跟踪值
    float v2;//跟踪值的微分
} TD_t;

//最速跟踪函数
float fst(float x1_delta, float x2, float r, float h0) {
    float d, y, a0, a, a1, a2;
    d = r * h0 * h0;
    a0 = h0 * x2;
    y = x1_delta + a0;
    a1 = sqrt(d * (d + 8 * fabs(y)));
    a2 = a0 + fsgn(y) * (a1 - d) / 2;
    a = (a0 + y) * fsg(y, d) + a2 * (1 - fsg(y, d));
    return (-r * (a / d) * fsg(a, d) - r * fsgn(a) * (1 - fsg(a, d)));
}
/**
 * @brief      : 与ADRC解耦的跟踪微分器
 * @attention  : 无
 * @return  {*}
 * @param {TD_t*} td td数据结构体
 */
void TDFunction_independent(TD_t* td) {
    float last_v1, last_v2;
    last_v1 = td->v1;
    last_v2 = td->v2;
    td->v1 = last_v1 + td->config.h * last_v2;
    td->v2 = last_v2 + td->config.h * fst(last_v1 - td->ref, last_v2, td->config.r, td->config.h0);
```

```
39    }
```

纯串环pid与TD+串环pid的控制效果对比如下：





> 关节1，$\frac{\pi}{4}$（45°）阶跃信号。
>
> 上图为纯串环pid，达到稳定约0.89s；下图为TD+串环pid，达到稳定约0.51s

2.1.3.2 底盘

2.1.3.2.1 离地检测

由于机器人需要根据是否离地来决定是否加载关节力矩前馈，我们还需要能够通过关节电机反馈力矩估计足端力。

为了方便底盘在维修架上的调试，此处将机器人的底盘基座视为固定基，并考虑连杆质量。考虑到连杆质量的动力学计算较为复杂，且连杆质量相对于底盘基座质量而言很小，故在正常运动状态将连杆质量忽略，即连杆1和连杆2的重力置零，仅使用虚功原理进行静力学计算。

下面是使用matlab计算由电机反馈力矩 $\tau 1, \tau 2$ 得到连杆2对轮的力 F_w 的方法。

代码块

```
1   syms L1 real %link1长
2   syms L2 real %link2长
3   syms q [2,1] real % 关节角
4   syms F_w [3,1] real %link2对轮的作用力
```

```
syms tau_w real %link2对轮的力矩
syms tau [2,1] real %抽象关节力矩
syms G1_ [3,1] real %link1的重力
syms G2_ [3,1] real %link2的重力
syms r_c1_ [3,1] real %link1质心位矢
syms r_c2_ [3,1] real %link2质心位矢
syms r_w [3,1] real %轮质心位矢
syms r_1_c1_ [3,1] real %link1质心在系1下的位矢
syms r_2_c2_ [3,1] real %link2质心在系2下的位矢

% 由于机体少有侧倾情况，为简化运算，此处指定重力在腿基坐标系z方向上为0
G1_(3) = 0;
G2_(3) = 0;
% 辅助函数，计算绕z轴旋转的旋转矩阵
R_z_fun = @(theta) [    cos(theta)    -sin(theta)    0;
                        sin(theta)    cos(theta)     0;
                        0             0              1];
% 0系到1系的旋转矩阵（相对于q1调整pi/2）
R_01 = R_z_fun(q(1) - pi/2);
% 1系到2系的旋转矩阵（包含z轴旋转和坐标系翻转）
R_12 = R_z_fun(q(2)) * sym([-1     0     0;
                        0      0     1;
                        0      1     0]);
% 0系到1系的平移向量（link1末端在基础坐标系中的位置）
t_01 = [L1*sin(q(1))    -L1*cos(q(1))   0]' ;
% 0系到2系的平移向量（link2末端/足端在基础坐标系中的位置）
t_02 = [L1*sin(q(1)) + L2*sin(q(1)+q(2))    -L1*cos(q(1)) - L2*cos(q(1)+q(2))
 0]';
% 0系到1系的齐次变换矩阵
T_01 = [    R_01    t_01;
            [0 0 0] 1   ];
% 0系到2系的齐次变换矩阵
T_02 = [    R_01*R_12   t_02;
            [0 0 0]     1   ];
% 通过齐次变换矩阵计算link1质心在世界坐标系中的位置
temp_c1 = ( T_01*[ r_1_c1_' 1]' );
r_c1_ = temp_c1(1:3);
% 通过齐次变换矩阵计算link2质心在世界坐标系中的位置
temp_c2 = ( T_02*[ r_2_c2_' 1]' );
r_c2_ = temp_c2(1:3);
% 轮轴点位置（link2末端位置）
r_w = t_02;

% 计算轮轴点位置对关节角的雅可比矩阵
J_rw_q = jacobian(r_w,q);
% 此处未知数数量为3，方程数为2，取最小范数解
```

```
50   F_w = J_rw_q*(J_rw_q'*J_rw_q)^(-1) * ( tau' + G1_'*jacobian(r_c1_,q) +
     G2_'*jacobian(r_c2_,q) - [tau_w tau_w] )';
51   F_w = simplify(F_w);
52   F_w_fun = matlabFunction(F_w)
53   ccode(F_w)
```

特别地，对于离地检测的场景，静力学正解的精度要求不高，因此我们仅计算连杆2对轮的力Fw而不是地对轮的力Fe。

当Fw小于特定的阈值，即可认为该腿离地。

2.1.3.2.2 平衡控制

算法介绍

四足平衡控制使用了IMU结合位置速度串级pid算法，下面的控制公式图引用自 https://www.bilibili.com/video/BV1jqGFz3ENH/。

俯仰角控制：

$$\theta'_{target} = kp_{\theta} \times (0 - \theta_{pitch})$$

期望角速度 | 系数 | 期望0角度 | 传感器角度

$$\Delta H = kp_{\theta'} \times (\theta'_{target} - \theta'_{pitch})$$

腿高调节值 | 系数 | 期望角速度 | 传感器角速度

$$H_{cur} = H_{init} + \sum_{t=1}^{\infty} \Delta H$$

新腿高坐标 | 原本腿高坐标 | 腿高调节值

滚转角控制：

$$\theta'_{target} = kp_{\theta} \times (0 - \theta_{roll})$$

期望角速度 | 系数 | 期望0角度 | 当前角度

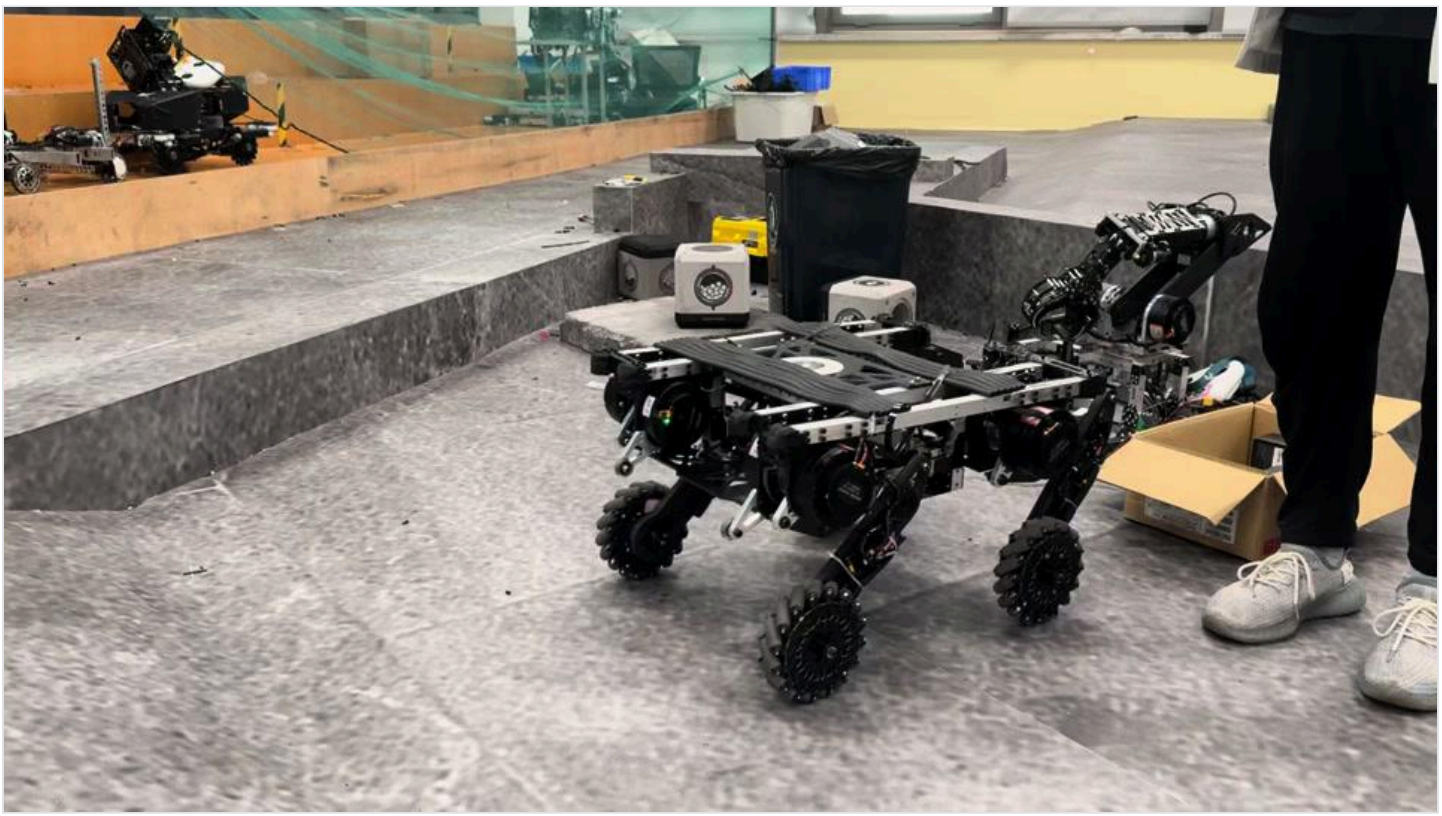
$$\Delta H = kp_{\theta'} \times (\theta'_{target} - \theta'_{roll})$$

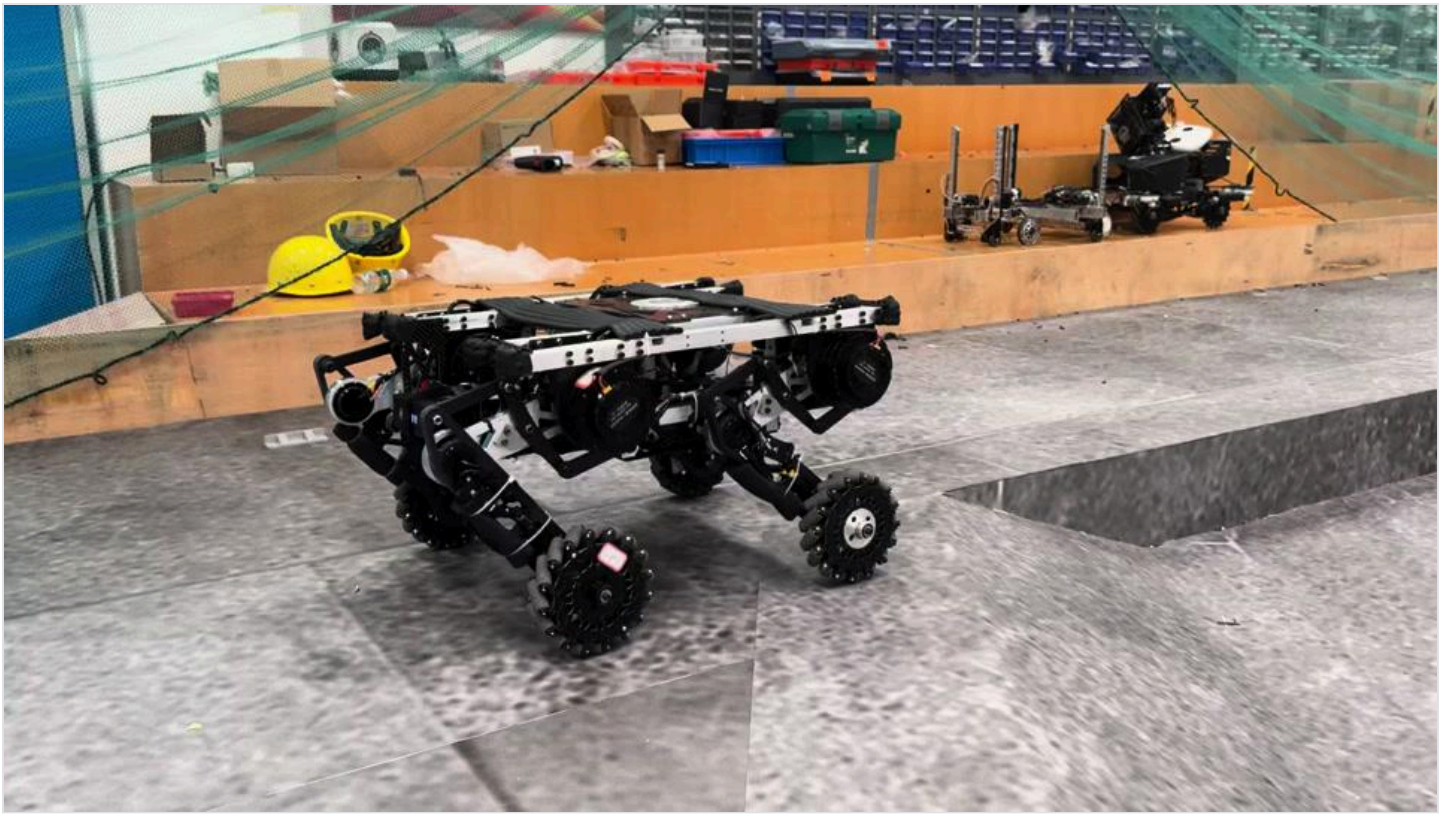
腿高调节值 | 系数 | 期望角速度 | 当前角速度

$$H_{cur} = H_{init} + \sum_{t=1}^{\infty} \Delta H$$

新腿高坐标 | 原本腿高坐标 | 腿高调节值

平衡控制效果演示





2.1.3.3 遥操作

为了对机械臂进行更精细更便捷的控制，不同于常见的同构型遥操作控制器，我们采用了Delta型并联机械臂式机器人+3编码器手柄分别控制六轴机械臂的末端xyz位置和夹爪rpy姿态，这种构型的遥操作控制器的优势在于能让操作手更自如地控制机械臂的工作姿态，避免了操作手需要同时控制多个关节角度而导致的不够灵活的问题，手柄还可以配备多样的按键实现模式切换。

2.1.3.3.1 Delta机构简化数学模型

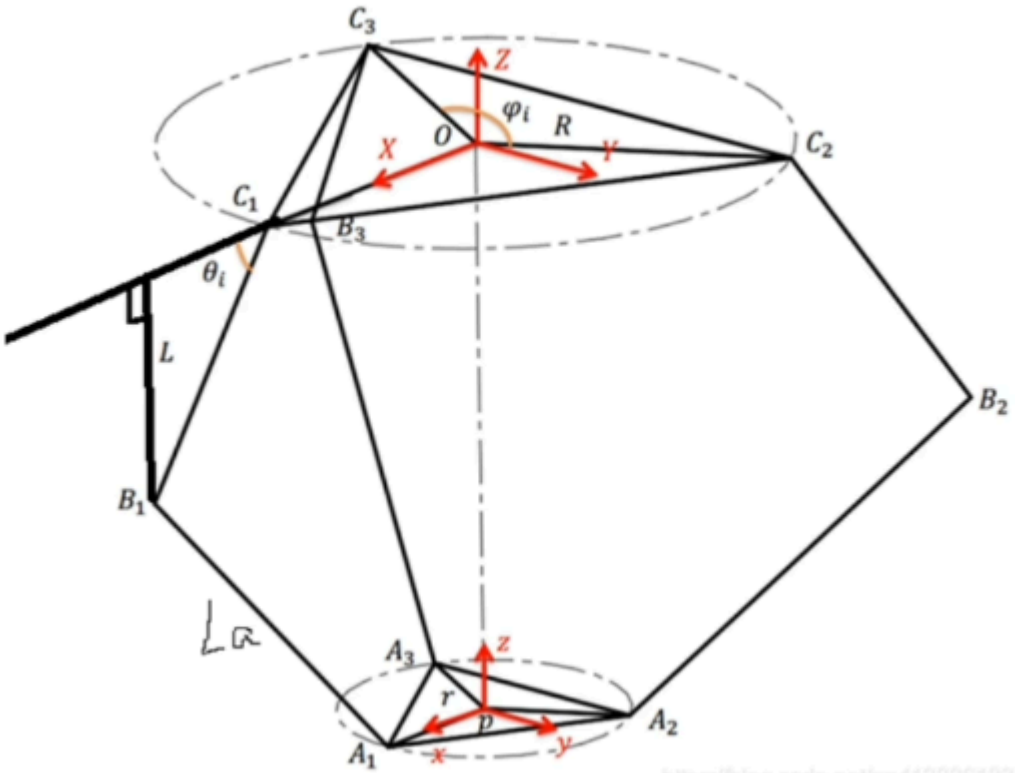
> 遥操作控制器的Delta机构参考了开源Delta X机器人的构型以及b站up革命的草鞋的相关教程

Delta机器人的优势在于结构简单，响应迅速。我们通过建立静平台（$O-XYZ$ 坐标系）中3个电机（置于 C_i)的旋转角度 θ_i 与动平台（$p-xyz$ 坐标系）原点坐标 $[x\ y\ z]^T$ 之间的数学关系，并将其按照比例映射到6轴机械臂的末端位置 $[X'\ Y'\ Z']^T$ ，实现对机械臂末端位置的控制。

模型参数表

结构参数	数学模型参数	含义
$\lvert OC_i\rvert$	R	静坐标系原点到静平台各顶点的长度
$\lvert pA_i\rvert$	r	动坐标系原点到动平台各顶点的长度
$\vec{op}$	$[x\ y\ z]^T$	动坐标系的原点相对于静坐标系的位置矢量
$\lvert C_iB_i\rvert$	L	主动臂（电机驱动的臂）的长度

$\|B_iA_i\|$	La	从动臂的长度
θ_i	θ_i	主动臂与静平台的夹角



> 上图截取自https://www.bilibili.com/video/BV1hv4y1K7SP/

2.1.3.3.2 正逆运动学解算

正运动学解算

代码块

```
disp("=============正运动学=============")
syms theta1 theta2 theta3 real
% 关节变量
% theta=[0 0 0].*pi/180
theta=[theta1 theta2 theta3];
% 验证值
% X=0
% Y=0
% Z=0

% 正运动学
A1=R + L*cos(theta(1)) - r;
```

```
B1 = 1;
C1=L*sin(theta(1));
A2=-(1.0/2.0)*(R+L*cos(theta(2))-r);
B2=(3^0.5 / 2)*(R+L*cos(theta(2))-r);
C2=L*sin(theta(2));
A3=-(1.0/2.0)*(R+L*cos(theta(3))-r);
B3=-(3^0.5 / 2)*(R+L*cos(theta(3))-r);
C3=L*sin(theta(3));
D1=(1.0/2.0)*(A2*A2-A1*A1+C2*C2-C1*C1+B2*B2);
A21=A2-A1;
C21=C2-C1;
D2=(1.0/2.0)*(A3*A3-A1*A1+C3*C3-C1*C1+B3*B3);
A31=A3-A1;
C31=C3-C1;
E1 = (B3*C21 - B2*C31)/(A21*B3 - A31*B2);
F1 = (B3*D1 - B2*D2)/(A21*B3 - A31*B2);
E2 = (A31*C21 - A21*C31)/(A31*B2 - A21*B3);
F2 = (A31*D1 - A21*D2)/(A31*B2 - A21*B3);
a = E1*E1 + E2*E2 + 1;
b = 2*E2*F2 + 2*C1 - 2*E1*(A1 - F1);
c = (A1 - F1)*(A1 - F1) + F2*F2 + C1*C1 - LS*LS;
Z = (-b - (b*b - 4*a*c)^0.5)/(2*a);
X = E1*Z + F1;
Y = E2*Z + F2;
X = simplify(X);
Y = simplify(Y);
Z = simplify(Z);

% 正雅可比
P=[X Y Z]'
ccode(P)
J=jacobian(P,theta');

% 匿名函数
x_fun=matlabFunction(X);
y_fun=matlabFunction(Y);
z_fun=matlabFunction(Z);
```

逆运动学解算

代码块

```
disp("============逆运动学============")
syms x y z real
% 末端基座
% x=127.6965
% y=0
```

```
% z=-311.4166
% 逆运动学
A1 = (x*x+y*y+z*z+L*L-LS*LS + (R-r)*(R-r)-2*x*(R-r))/(2*L);
B1 = -(R - r - x);
C1 = z;
A2 = (x*x+y*y+z*z+L*L-LS*LS + (R-r)*(R-r)+(x - sqrt(3)*y)*(R-r))/L;
B2 = -2*(R - r)-(x - sqrt(3)*y);
C2 = 2*z;
A3 = (x*x+y*y+z*z+L*L-LS*LS + (R-r)*(R-r)+(x + sqrt(3)*y)*(R-r))/L;
B3 = -2*(R - r)-(x + sqrt(3)*y);
C3 = 2*z;
K1 = A1 + B1;
U1 = 2 * C1;
V1 = A1 - B1;
K2 = A2 + B2;
U2 = 2 * C2;
V2 = A2 - B2;
K3 = A3 + B3;
U3 = 2 * C3;
V3 = A3 - B3;
theta1 = 2*(atan((-U1-sqrt(U1*U1 - 4 * K1 * V1)) / (2 * K1)));
theta2 = 2*(atan((-U2 - sqrt(U2 * U2 - 4 * K2 * V2)) / (2 * K2)));
theta3 = 2*(atan((-U3 - sqrt(U3 * U3 - 4 * K3 * V3)) / (2 * K3)));

% 匿名函数
t1_fun=matlabFunction(theta1)
t2_fun=matlabFunction(theta2)
t3_fun=matlabFunction(theta3)
```

2.1.3.3.3 静力学解算及重力补偿

使用虚功原理进行计算，选取主动力为电机输出力矩 τ 与末端手柄对delta动平台的力 $-F$：

代码块

```
disp("============静力学============")
syms F_x F_y F_z real

F = [F_x F_y F_z]';
% 雅可比矩阵 J 的求解见上方正运动学解算部分
touque=J' * F
ccode(touque(1))
ccode(touque(2))
ccode(touque(3))
simplify(touque)

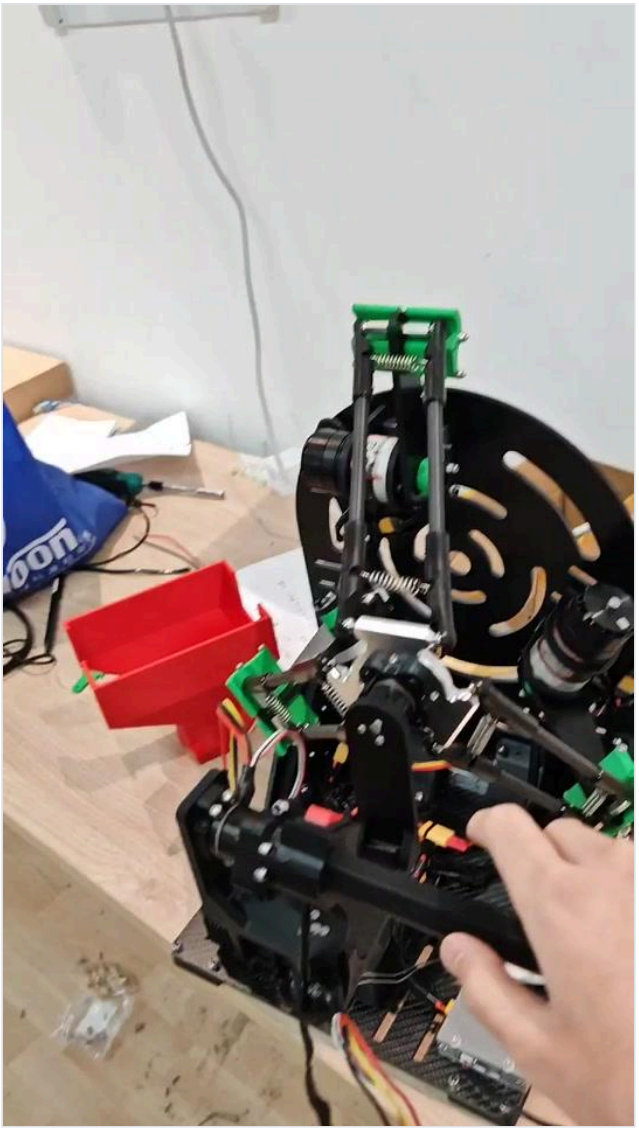
% 匿名函数
```

```
13   touque_fun=matlabFunction(touque)
```

Delta机器人控制效果如下：



重力补偿效果展示



2.1.3.3.4 遥操作控制流程

总体而言，我们的遥操作控制器采用分模块控制方式，实现对六轴机械臂位置与姿态的全面调控：通过Delta机构控制机械臂末端的xyz位置，通过自主设计的3编码器手柄控制机械臂末端夹爪的rpy姿态；此外，手柄还可配置夹爪开合等多样化按键拓展功能，相关细节在此不再赘述。

当机器人通过无线链路接收到遥操作控制器发送的数据后，首先通过TD跟踪微分器对控制信号进行处理，实现信号平滑。这一处理一方面能在一定程度上弥补遥操作控制器低帧率导致的控制信号不平滑问题，另一方面可有效防止潜在丢包引发的信号跳变，避免其对机械臂结构造成损害及产生安全隐患。

信号处理完成后，需对Delta机构坐标下的 $[x\ y\ z]^T$ 与机械臂的实际工作区间进行比例映射，其中映射比例 $MAPPINGRATIO$ 可自由调整，以平衡控制精度与控制速度。同时，若遥操作控制器发出的控制指令要求机械臂到达其工作区间外的位置，我们将通过底盘位置控制，精确计算并补偿机械臂伸展范围不足的部分，从而实现机械臂工作区间的拓展：

代码块

```
case arm_ctrl_cartesian:
// 末端空间位置控制模式
{
```

```
    if (obj->custom_controller_flag == 1) {
        // 使用遥操作控制器
        // 将遥操作控制器的xyz和rpy转化为齐次变换矩阵，方便后续计算
        obj->arm_cmd.T_end = RP_To_T(RPY_TO_R(obj->custom_recv_data.gripper_rpy),
            obj->custom_recv_data.xyz_fdb_base);

        // 目标xyz
        float target_x =
            (obj->arm_cmd.T_end.T[0][3] - obj->custom_xyz_offset[0]) * MAPPING_RATIO_X
            + obj->robot_xyz_offset[0];
        float target_y =
            (obj->arm_cmd.T_end.T[1][3] - obj->custom_xyz_offset[1]) * MAPPING_RATIO_Y
            + obj->robot_xyz_offset[1];
        float target_z =
            (obj->arm_cmd.T_end.T[2][3] - obj->custom_xyz_offset[2]) * MAPPING_RATIO_Z
            + obj->robot_xyz_offset[2];
        // 限幅 + 底盘补偿
        // x方向
        if (target_x > ROBOT_X_MAX) {
            obj->arm_cmd.T_end.T[0][3] = ROBOT_X_MAX;
            obj->chassis_control.x_ref +=
                (target_x - ROBOT_X_MAX - obj->arm_xy_comp[0]) * 1000;
            obj->arm_xy_comp[0] = target_x - ROBOT_X_MAX;
        } else if(target_x < ROBOT_X_MIN) {
            obj->arm_cmd.T_end.T[0][3] = ROBOT_X_MIN;
            obj->chassis_control.x_ref +=
                (target_x - ROBOT_X_MIN - obj->arm_xy_comp[0]) * 1000;
            obj->arm_xy_comp[0] = target_x - ROBOT_X_MIN;
        } else {
            obj->arm_cmd.T_end.T[0][3] = target_x;
        }
        // y方向
        if (target_y > ROBOT_Y_MAX) {
            obj->arm_cmd.T_end.T[1][3] = ROBOT_Y_MAX;
            obj->chassis_control.y_ref +=
                (target_y - ROBOT_Y_MAX - obj->arm_xy_comp[1]) * 1000;
            obj->arm_xy_comp[1] = target_y - ROBOT_Y_MAX;
        } else if(target_y < ROBOT_Y_MIN) {
            obj->arm_cmd.T_end.T[1][3] = ROBOT_Y_MIN;
            obj->chassis_control.y_ref +=
                (target_y - ROBOT_Y_MIN - obj->arm_xy_comp[1]) * 1000;
            obj->arm_xy_comp[1] = target_y - ROBOT_Y_MIN;
        } else {
```

```
            obj->arm_cmd.T_end.T[1][3] = target_y;
        }
        // z方向
        obj->arm_cmd.T_end.T[2][3] = fclip(target_z, ROBOT_Z_MIN, ROBOT_Z_MAX);
    }
```

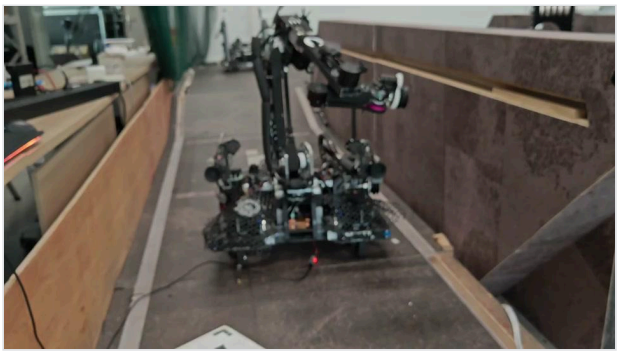
夹爪姿态控制方面，通过读取手柄的三编码器数值，经过坐标变换得到机械臂末端的旋转矩阵 R_6^0 ，完成对机械臂末端姿态的精准控制：

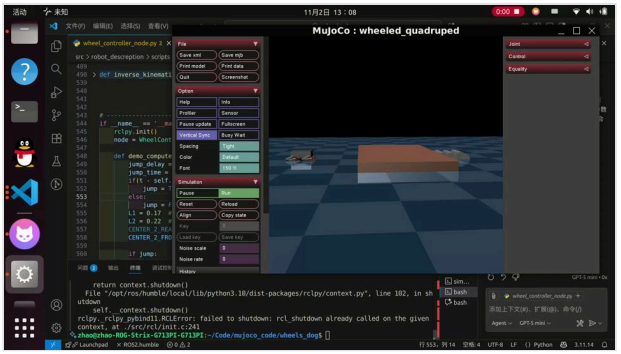
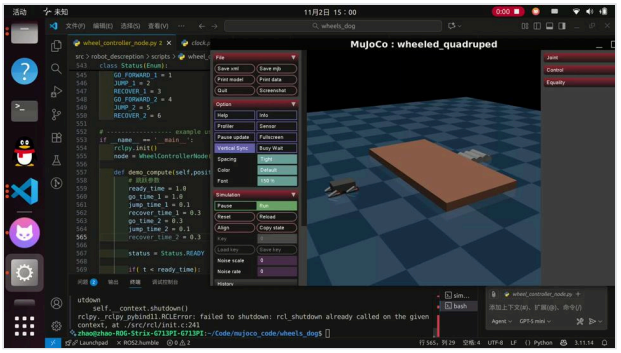
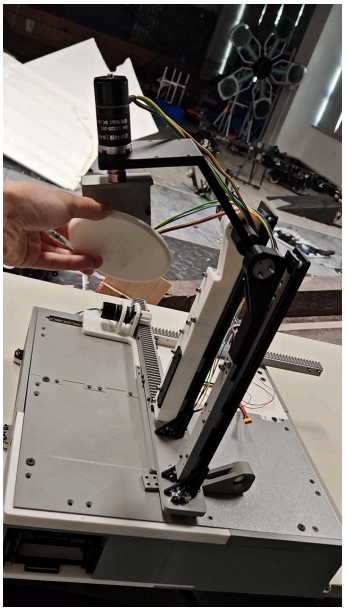
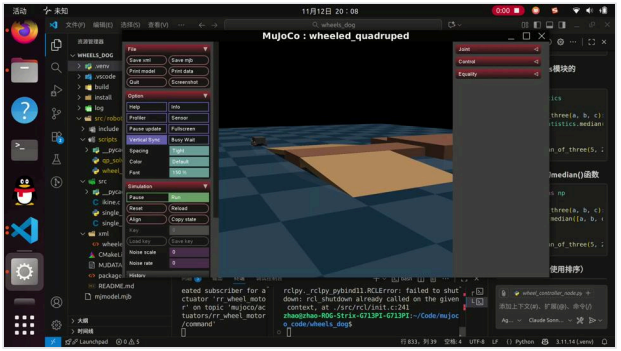
代码块

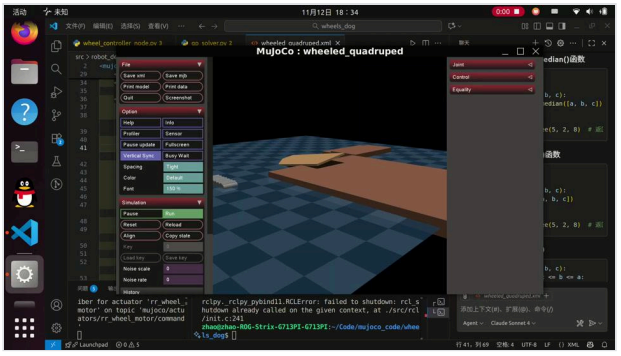
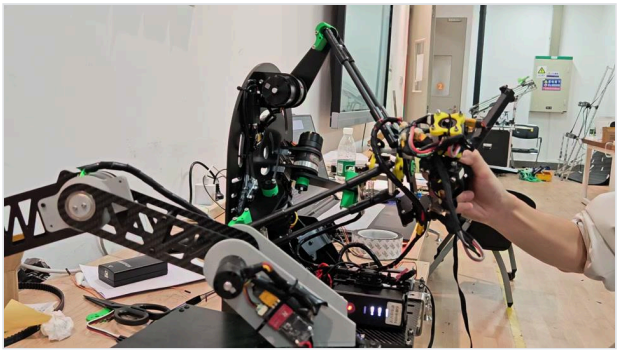
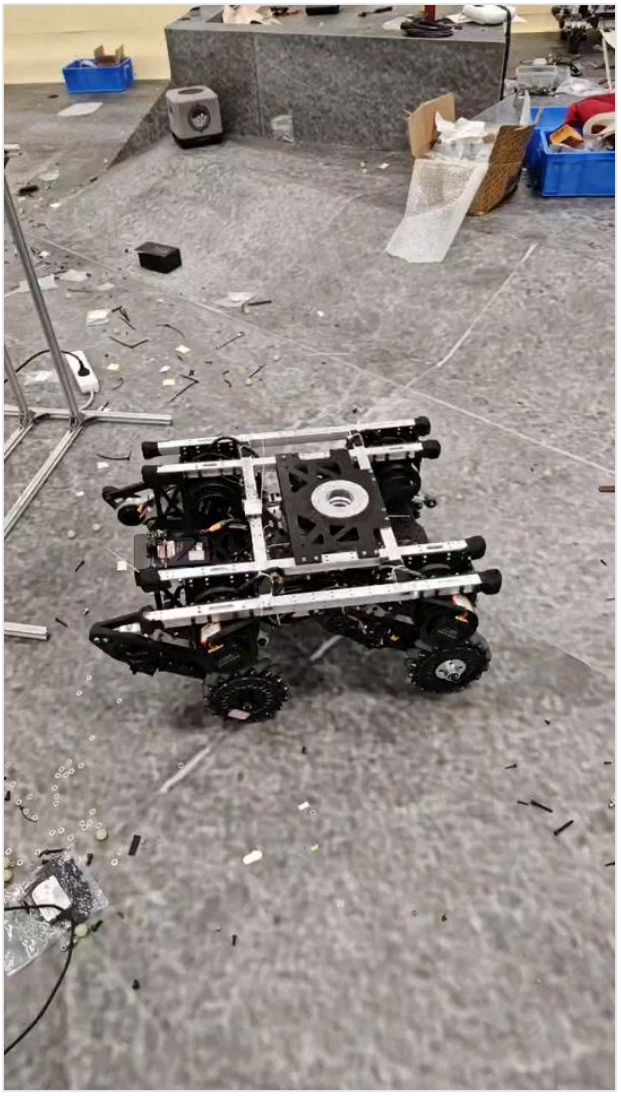
```
// 转换到基座坐标系
Mut_33_33(R_RPY_TO_BASE.R, T_To_R(Serial_Link_FKine(obj->rpy_link, q)).R,
    obj->rpy_fdb_base.R);
// 转换为机械臂末端姿态
Mut_33_33(obj->rpy_fdb_base.R, R_CUSTOM_TO_ROBOT.R, obj->gripper_R.R);
// 转换为rpy角发送，节省带宽
obj->gripper_rpy = R_TO_RPY(obj->gripper_R);
```

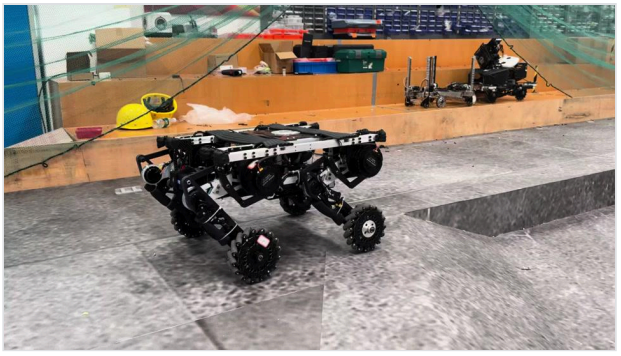
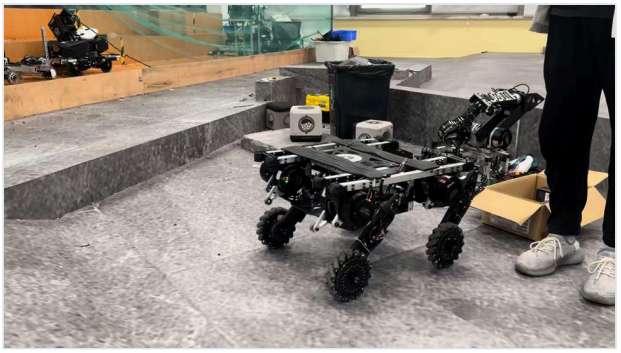
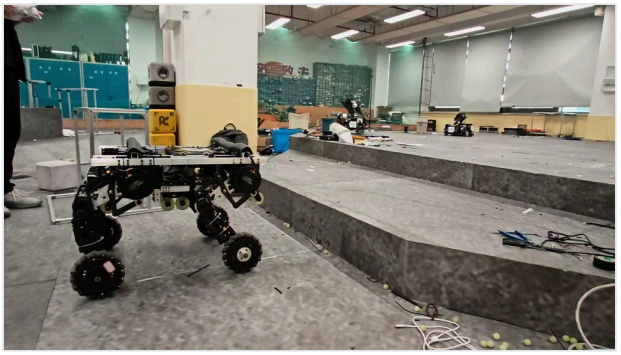
至此，遥操作控制器实现了对机械臂所有位置与姿态的完整控制。

3.1 版本迭代过程记录

版本号或阶段	***功能或性能详细说明***	***图片/视频***	***完成时间***
V1.1	通过动力学建模与轨迹规划算法，实现了机械臂随机姿态的平滑运动规划		2025.9.13
V1.1	遥操作控制器xyz传动与pry三轴初步布局与实现方案确定，末端pry尺寸与电控走线方案待修改		2025.10.17

V1.1	在仿真平台上实现了四轮足底盘如下功能：静力学补偿；跳跃上200mm台阶；稳定通过70mm减速带	 	2025.11.2
V1.1	初版遥操作控制器组装完成		2025.11.16
V1.1	在仿真平台上实现了四轮足底盘姿态平衡控制，实现如下功能： 过单边台阶； 坡道自平衡		2025.11.12

			
V1.1	第二版遥操作控制器装配完成，正逆解算验证通过		2025.12.21
V1.1	四轮足底盘装配完成，基本运动功能实现		2026.2.1
V1.1	四轮足底盘实现 过单边台阶、坡道自平衡		2026.2.6

		 	
V1.1	上双级台阶实现		2026.3.16